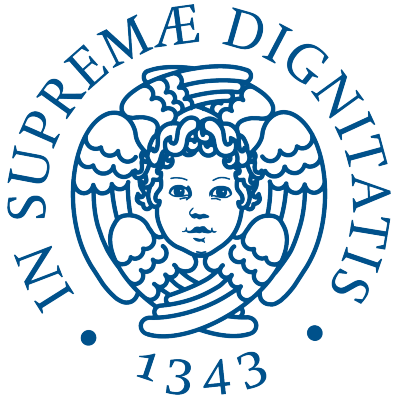

University of Pisa
Master's Degree in Computer Science,
Artificial Intelligence

Computational Mathematics for Learning & Data Analysis (646AA)

Group number: 63
Project number: 25 ML

Members:
Matthew Bernardi
Gabriele Marsili

Academic Year: 2025/2026

Project statement (verbatim).

25. (P) is so-called *extreme learning*, i.e., a neural network with one hidden layer, $y = w\sigma(W_1 x)$, where the weight matrix for the hidden layer W_1 is a fixed random matrix, $\sigma(\cdot)$ is an elementwise activation function of your choice, and the output weight vector w is chosen by solving a least-squares problem with L_1 regularization $\arg\min_w f(w)$, with $f(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|w\|_1$.

(A1) is *iteratively reweighted least squares*: i.e., an iteration where you solve at each step the least-squares problem $w_{k+1} = \arg\min_w f_k(w)$, $f_k(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|W_k w\|_2^2$ where W_k is the diagonal matrix with entries $(W_k)_{ii} = |(w_k)_i|^{-1/2}$.

(A2) is an algorithm of the class of the *deflected subgradient methods*.

Contents

Chapter 1

Introduction to the problem

1.1 Problem 25 ML

25. (P) is so-called *extreme learning*, i.e., a neural network with one hidden layer, $y = w\sigma(W_1 x)$, where the weight matrix for the hidden layer W_1 is a fixed random matrix, $\sigma(\cdot)$ is an elementwise activation function of your choice, and the output weight vector w is chosen by solving a least-squares problem with L_1 regularization $\arg\min_w f(w)$, with $f(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|w\|_1$.

(A1) is *iteratively reweighted least squares*: i.e., an iteration where you solve at each step the least-squares problem

$$w_{k+1} = \arg\min_w f_k(w), \quad f_k(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|W_k w\|_2^2$$

where W_k is the diagonal matrix with entries $(W_k)_{ii} = |(w_k)_i|^{-1/2}$.

(A2) is an algorithm of the class of the *deflected subgradient methods*.

1.2 Extreme Learning Machine

Extreme Learning Machine (ELM) [25] is a machine learning model implemented as a neural network with a single hidden layer, in which:

- $\mathbf{W_1} \in \mathbb{R}^{H \times N}$ is the input-to-hidden weight matrix (where H is the number of nodes in the hidden layer and N is the number of input nodes) that is **randomly generated** and **static** (does not change during training) [25].
- $\sigma(\cdot)$ is the element-wise (nonlinear) activation function applied to the $\mathbf{W_1 x} \in \mathbb{R}^H$ vector, where $\mathbf{x} \in \mathbb{R}^N$ is the input vector. The choice of using an activation function for the hidden layer results, under mild assumptions and with high probability, in the hidden layer feature matrix with full rank when $M \geq H$, since random projections through a nonlinear function produce linearly independent features.

- $\mathbf{w} \in \mathbb{R}^H$ is the hidden-to-output weight vector, chosen by solving the least-squares problem.
- $\hat{y} = \mathbf{w}^T \sigma(\mathbf{W_1 x}) \in \mathbb{R}$ is the prediction of the model.

Once $\mathbf{W_1}$ is fixed, the hidden-layer activations $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}} \mathbf{W}_1^T)$ are constant, and finding $\mathbf{w}$ reduces to solving the Least-Squares problem described in the following section.

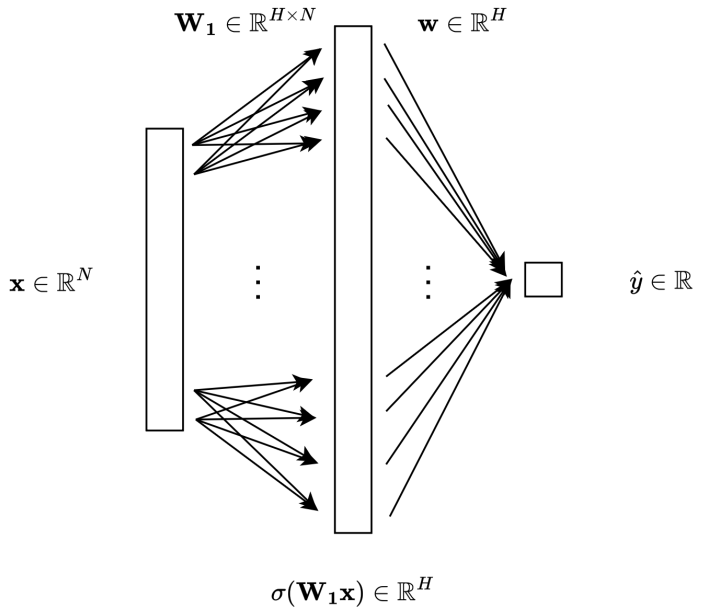

Figure 1.1: Visual representation of an Extreme Learning Machine model.

1.3 Least-Squares Problem

Considering $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}} \mathbf{W}_1^T) \in \mathbb{R}^{M \times H}$ the matrix of activations of the hidden layer, while $\mathbf{X}_{\text{origin}} \in \mathbb{R}^{M \times N}$ is the matrix of all the inputs, and $\mathbf{y} \in \mathbb{R}^M$ is the vector of all the target values, the definition of the Least Squares Problem [6] in the ELM case is the following:

$$\min_{\mathbf{w}} ||\mathbf{X}\mathbf{w} - \mathbf{y}||_2^2 \tag{1.1}$$

The Least-Squares problem always admits at least one solution, and the solution is unique if and only if the matrix $\mathbf{A}$ has full column rank. In that case, we can write the optimization problem as follows:

$$\min_{\mathbf{w}} \frac{1}{2}\mathbf{w}^T \mathbf{X}^T \mathbf{X}\mathbf{w} - \mathbf{y}^T \mathbf{X}\mathbf{w} + \frac{1}{2}\mathbf{y}^T \mathbf{y} \tag{1.2}$$

since the gradient of this function is $\mathbf{X}^T\mathbf{X}\mathbf{w} - \mathbf{X}^T\mathbf{y}$, the Hessian matrix is $\mathbf{X}^T\mathbf{X} \succ 0$, which means that the function is strictly convex. In this case, the minimum exists and is unique, and can be found solving the equation:

$$\mathbf{X}^T\mathbf{X}\mathbf{w} - \mathbf{X}^T\mathbf{y} = 0$$

or

$$\mathbf{X}^T\mathbf{X}\mathbf{w} = \mathbf{X}^T\mathbf{y}$$

where $\mathbf{X}^T\mathbf{X}$ is square invertible since it's positive definite. The linear system has a unique solution $\mathbf{w}_0 = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$. However, adding the L_1 regularization term (Section 1.4) destroys the closed-form structure, requiring iterative methods.

1.4 Regularization Techniques

1.4.1 L_1 regularization

When adding the L_1 regularization term to the Least-Squares problem in the case of ELM, the equation is the following:

$$\min_{\mathbf{w}} f(\mathbf{w}) = \frac{1}{2}||\mathbf{X}\mathbf{w} - \mathbf{y}||_2^2 + \lambda_{\text{LASSO}} \sum_{i}^{H} |w_i| \tag{1.3}$$

The price to pay is the non-differentiability of $f(\mathbf{w})$ at any point where some $w_i = 0$, which prevents direct use of gradient-based methods.

Because of that, we can use the Deflected Subgradient Methods (2) and the IRLS algorithm (2), which we will see in the following chapters.

1.5 Objective Function Analysis

The full objective function we need to minimize is the Lasso regularization of the Least-Squares problem:

$$\min_{\mathbf{w}} f(\mathbf{w}) = \underbrace{\frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2}_{g(\mathbf{w})} + \underbrace{\lambda_{\text{LASSO}}\|\mathbf{w}\|_1}_{h(\mathbf{w})}$$

It is useful to analyse the mathematical properties of $f(\mathbf{w})$, as they determine which algorithms can be applied.

1.5.1 Convexity

$f(\mathbf{w})$ is the sum of two convex functions [2, p. 67]. The quadratic term $g(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$ has Hessian $\nabla^2 g(\mathbf{w}) = \mathbf{X}^T\mathbf{X} \succeq 0$, making it convex. If $\mathbf{X}$ has full column rank, then $\mathbf{X}^T\mathbf{X} \succ 0$ and g is *strictly convex*. The regularization

term $h(\mathbf{w}) = \lambda_{\text{LASSO}}\|\mathbf{w}\|_1$ is convex as a scaled norm. Therefore $f = g + h$ is convex; and if $\mathbf{X}$ has full column rank, f is **strictly convex** and admits a **unique global minimum**.

Proposition 1.1. *If $\mathbf{X} \in \mathbb{R}^{M\times H}$ has full column rank, then $f(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda_{LASSO}\|\mathbf{w}\|_1$ is strictly convex and admits a unique global minimizer $\mathbf{w}^*$ [2, p. 73].*

Proof. We write $f = g+h$ where $g(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w}-\mathbf{y}\|_2^2$ and $h(\mathbf{w}) = \lambda_{\text{LASSO}}\|\mathbf{w}\|_1$.

Strict convexity. Since $\mathbf{X}$ has full column rank, for any $\mathbf{v} \neq \mathbf{0}$ we have $\mathbf{X}\mathbf{v} \neq \mathbf{0}$, so $\mathbf{v}^T(\mathbf{X}^T\mathbf{X})\mathbf{v} = \|\mathbf{X}\mathbf{v}\|_2^2 > 0$. Hence the Hessian of g is $\nabla^2 g = \mathbf{X}^T\mathbf{X} \succ 0$, making g strictly convex. h is convex as a non-negative multiple of a norm. The sum of a strictly convex function and a convex function is strictly convex [12], so f is strictly convex.

Existence and uniqueness of the minimizer. Since $\mathbf{X}^T\mathbf{X} \succ 0$, g is coercive: $g(\mathbf{w}) \to +\infty$ as $\|\mathbf{w}\| \to \infty$. Because $h \geq 0$, $f = g + h$ is also coercive. This means the sub-level set $\{\mathbf{w} : f(\mathbf{w}) \leq f(\mathbf{0})\}$ is compact, and since f is continuous, it attains its minimum on this set. Strict convexity then rules out two distinct minimizers: if $\mathbf{w}_1 \neq \mathbf{w}_2$ both minimized f, the midpoint $\frac{\mathbf{w}_1+\mathbf{w}_2}{2}$ would give a strictly smaller value, a contradiction. □

1.5.2 Non-smoothness

Unlike the quadratic term, the L_1 penalty $h(\mathbf{w}) = \lambda_{\text{LASSO}}\|\mathbf{w}\|_1$ is **not differentiable** [18] at any point where some component $w_i = 0$. This means f itself is non-smooth, and we cannot rely on gradient-based methods alone. At points where all $w_i \neq 0$, the gradient exists and is:

$$\nabla f(\mathbf{w}) = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda_{\text{LASSO}}\,\text{sign}(\mathbf{w})$$

where $\text{sign}(\mathbf{w})$ is applied element-wise. At points where some $w_i = 0$, one must reason in terms of *subgradients* (see Chapter 2).

1.5.3 Optimality conditions

Since f is non-smooth, the classical condition $\nabla f(\mathbf{w}^*) = 0$ does not apply at every minimizer. Instead, the optimality condition is expressed via the **subdifferential** [14]: $\mathbf{w}^*$ is a global minimizer for our ELM problem if and only if:

$$0 \in \partial f(\mathbf{w}^*) = \mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + \lambda_{\text{LASSO}}\,\partial\|\mathbf{w}^*\|_1$$

where the subdifferential of the L_1 norm is given component-wise by:

$$[\partial\|\mathbf{w}\|_1]_i = \begin{cases} \{+1\} & \text{if } w_i > 0, \\ \{-1\} & \text{if } w_i < 0, \\ [-1,+1] & \text{if } w_i = 0. \end{cases}$$

This condition can be written component-wise as: for each $i = 1, \ldots, H$,

$$\left[\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y})\right]_i + \lambda_{\text{LASSO}} s_i = 0, \qquad s_i \in \partial|w_i^*|. \tag{1.4}$$

Intuitively, λ_{LASSO} controls the sparsity of $\mathbf{w}^*$: the larger λ_{LASSO} is, the easier it is for the optimality condition to force components $w_i^* = 0$, producing a sparser solution.

Chapter 2

Algorithm 1: Iteratively Reweighted Least Squares

Iteratively Reweighted Least Squares (IRLS) [8] is a classical algorithm for solving optimization problems involving non-smooth objective functions of the form of a p-norm. Its core idea is to handle non-smoothness by solving a sequence of *weighted* least-squares problems, each of which is smooth and admits an efficient closed-form solution. In our setting, IRLS is used to minimize the L_1-regularized least-squares objective:

$$\min_{\mathbf{w}\in\mathbb{R}^H} f(\mathbf{w}) = \underbrace{\frac{1}{2}\|\mathbf{X}\mathbf{w}-\mathbf{y}\|_2^2}_{g(\mathbf{w})} + \underbrace{\lambda_{\text{LASSO}}\|\mathbf{w}\|_1}_{h(\mathbf{w})} \tag{2.1}$$

where $\mathbf{X}\in\mathbb{R}^{M\times H}$, $\mathbf{y}\in\mathbb{R}^M$, and $\lambda_{\text{LASSO}} > 0$ is the regularization hyperparameter of the original problem.

2.1 The core idea: a strict upper bound for the L_1 norm

The difficulty in (2.1) is the L_1 term: convex but non-differentiable at every component crossing zero. To replace it with a smooth surrogate we use the elementary inequality $(|w_i| - |(w_k)_i|)^2 \geq 0$, which after expansion and division by $2|(w_k)_i|$ gives the majorization

$$|w_i| \leq \frac{w_i^2}{2|(w_k)_i|} + \frac{|(w_k)_i|}{2}, \qquad (w_k)_i \neq 0. \tag{2.2}$$

Summing over $i = 1,\ldots,H$ yields the bound on the L_1 norm

$$\|\mathbf{w}\|_1 \leq \frac{1}{2}\mathbf{w}^T\mathbf{W}_k^T\mathbf{W}_k\mathbf{w} + \frac{1}{2}\|\mathbf{w}_k\|_1 \tag{2.3}$$

where $\mathbf{W}_k \in \mathbb{R}^{H\times H}$ is a diagonal matrix with entries:

$$(\mathbf{W}_k)_{ii} = |(w_k)_i|^{-1/2} \tag{2.4}$$

This bounds the non-smooth L_1 term using a *smooth, quadratic* expression $\frac{1}{2}\mathbf{w}^T\mathbf{W}_k^T\mathbf{W}_k\mathbf{w}$, plus a constant term that depends only on the current iterate $\mathbf{w}_k$. Intuitively, $\mathbf{W}_k^T\mathbf{W}_k$ implements a *personalized penalization* of each component of $\mathbf{w}$ [5]: components with small magnitude at iteration k receive a large weight $|(w_k)_i|^{-1}$ and are strongly pushed toward zero, while components with large magnitude receive a small weight and are left relatively free. This *reweighting* mechanism is what drives IRLS to produce sparse iterates, mimicking the sparsity-inducing effect of the L_1 norm through a sequence of smooth quadratic problems.

Handling near-zero components. Equation (2.4) becomes numerically unstable when $(w_k)_i \approx 0$, since $|(w_k)_i|^{-1/2}$ can blow up. To prevent division by (near) zero, a **safety threshold** $\varepsilon_{\text{thr}} > 0$ is introduced:

$$(\mathbf{W}_k)_{ii} = \Big(\max\big(|(w_k)_i|,\, \varepsilon_{\text{thr}}\big)\Big)^{-1/2} \tag{2.5}$$

Typical values are $\varepsilon_{\text{thr}} \in [10^{-6}, 10^{-10}]$.

2.2 The Majorization-Minimization interpretation

IRLS fits naturally inside the Majorization-Minimization (MM) framework [26]. Instead of minimizing f directly, at each iteration k we build a *surrogate* $Q(\mathbf{w}, \mathbf{w}_k)$ that (i) majorizes f, i.e. $Q(\mathbf{w}, \mathbf{w}_k) \geq f(\mathbf{w})$ for all $\mathbf{w}$; (ii) touches f at $\mathbf{w}_k$, i.e. $Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$; and (iii) is smooth, so that it can be minimized in closed form. Plugging the quadratic upper bound (2.3) into (2.1) gives our surrogate,

$$Q(\mathbf{w}, \mathbf{w}_k) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{W}_k\mathbf{w}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1\,, \tag{2.6}$$

which is indeed a quadratic in $\mathbf{w}$. The majorization property is built in via (2.2). The touching property follows from a short computation: at $\mathbf{w} = \mathbf{w}_k$, each summand of the quadratic penalty becomes $(\mathbf{W}_k)_{ii}^2(w_k)_i^2 = (w_k)_i^2/|(w_k)_i| = |(w_k)_i|$, so

$$Q(\mathbf{w}_k, \mathbf{w}_k) = \tfrac{1}{2}\|\mathbf{X}\mathbf{w}_k - \mathbf{y}\|_2^2 + \tfrac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1 + \tfrac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1 = f(\mathbf{w}_k).$$

Taking $\mathbf{w}_{k+1} = \arg\min_{\mathbf{w}} Q(\mathbf{w}, \mathbf{w}_k)$, the two MM properties combined give $f(\mathbf{w}_{k+1}) \leq Q(\mathbf{w}_{k+1}, \mathbf{w}_k) \leq Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$, so the sequence $\{f(\mathbf{w}_k)\}$ is non-increasing. IRLS thus guarantees a monotone decrease of the objective in exact arithmetic; in floating-point arithmetic the monotonicity is preserved up to the accuracy of the linear solver used for the subproblem.

2.3 The weighted least-squares subproblem

To find the next iterate $\mathbf{w}_{k+1}$, we must minimize the surrogate function $Q(\mathbf{w}, \mathbf{w}_k)$ with respect to $\mathbf{w}$:

$$\mathbf{w}_{k+1} = \arg\min_{\mathbf{w}} \left(\frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{W}_k\mathbf{w}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1 \right)$$

Since the term $\frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1$ is constant with respect to $\mathbf{w}$, it does not affect the optimization and can be dropped entirely. The minimization problem simplifies to a standard **weighted least-squares problem with L_2 regularization** (Ridge regression):

$$\mathbf{w}_{k+1} = \arg\min_{\mathbf{w}} \left(\frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda_{\text{IRLS}}\|\mathbf{W}_k\mathbf{w}\|_2^2 \right) \tag{2.7}$$

provided that we strictly define the IRLS regularization parameter as:

$$\lambda_{\text{IRLS}} = \frac{1}{2}\lambda_{\text{LASSO}} \tag{2.8}$$

Relation to the LASSO regularization parameter.
The factor $1/2$ in (2.8) is forced by the majorization constant in (2.3). The upper bound on $\|\mathbf{w}\|_1$ carries a coefficient $\frac{1}{2}$ in front of the quadratic form $\mathbf{w}^T\mathbf{W}_k^T\mathbf{W}_k\mathbf{w}$, so the penalty entering the surrogate (2.6) is $\frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{W}_k\mathbf{w}\|_2^2$. Rewriting this as $\lambda_{\text{IRLS}}\|\mathbf{W}_k\mathbf{w}\|_2^2$ in (2.7) therefore requires $\lambda_{\text{IRLS}} = \frac{1}{2}\lambda_{\text{LASSO}}$. This is precisely the factor that makes the fixed point of IRLS coincide with the LASSO optimum, as shown in the proof of Theorem 2.2.

Proposition 2.1 (Weighted normal equations)**.** *The unique solution of* (2.7) *satisfies:*

$$\left(\mathbf{X}^T\mathbf{X} + 2\lambda_{IRLS}\mathbf{W}_k^T\mathbf{W}_k\right)\mathbf{w}_{k+1} = \mathbf{X}^T\mathbf{y} \tag{2.9}$$

Proof. The gradient of the objective in (2.7) is:

$$\nabla f_k(\mathbf{w}) = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k\mathbf{w}$$

Setting this to zero yields (2.9). The coefficient matrix $\mathbf{X}^T\mathbf{X} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$ is symmetric positive definite (it is the sum of $\mathbf{X}^T\mathbf{X} \succeq 0$ and $2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k \succ 0$ since $\varepsilon_{\text{thr}} > 0$), so the system has a unique solution. □

Since $\mathbf{W}_k$ is diagonal, defining $\mathbf{Q}_k = \mathbf{X}^T\mathbf{X} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$ and $\mathbf{b} = \mathbf{X}^T\mathbf{y}$, equation (2.9) reduces to:

$$\mathbf{Q}_k\,\mathbf{w}_{k+1} = \mathbf{b} \tag{2.10}$$

This is an $H \times H$ symmetric positive definite (SPD) linear system, which can be solved efficiently at each iteration. Note that $\mathbf{b} = \mathbf{X}^T\mathbf{y}$ can be pre-computed once before the iterative loop, since it does not depend on k. Similarly, $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ is pre-computed once, and only the diagonal $2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$ is updated at each iteration.

Solving the linear system. Because $\mathbf{Q}_k$ is SPD, two efficient methods can be used:

- **Cholesky factorization:** [28] computes $\mathbf{Q}_k = \mathbf{L}\mathbf{L}^T$ in $O(H^3/3)$ operations, then solves two triangular systems in $O(H^2)$. This is the direct method of choice for moderate-sized problems.
- **Conjugate Gradient (CG):** [28] an iterative method that converges in at most H steps in exact arithmetic, with convergence rate depending on the condition number $\kappa(\mathbf{Q}_k)$. Preferable for large-scale problems where H is too large for Cholesky.

2.4 The complete algorithm

Algorithm 1 Iteratively Reweighted Least Squares (IRLS)

Require: $\mathbf{X} \in \mathbb{R}^{M \times H}$, $\mathbf{y} \in \mathbb{R}^M$, $\lambda_{\text{LASSO}} > 0$, $\varepsilon_{\text{thr}} > 0$, $\varepsilon_{\text{stop}} > 0$, $k_{\max}$

Pre-compute $\mathbf{A} \leftarrow \mathbf{X}^T\mathbf{X}$, $\mathbf{b} \leftarrow \mathbf{X}^T\mathbf{y}$
Define $\lambda_{\text{IRLS}} \leftarrow \frac{1}{2}\lambda_{\text{LASSO}}$
Initialize $\mathbf{w}_0 \leftarrow \mathbf{A}^{-1}\mathbf{b}$ (ordinary least-squares solution)
for $k = 0, 1, 2, \ldots, k_{\max}$ **do**
    Compute weights: $(\mathbf{W}_k)_{ii} \leftarrow \big(\max(|(w_k)_i|, \varepsilon_{\text{thr}})\big)^{-1/2}$ for all i
    Assemble system: $\mathbf{Q}_k \leftarrow \mathbf{A} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$
    Solve: $\mathbf{Q}_k\,\mathbf{w}_{k+1} = \mathbf{b}$ *(via Cholesky or CG)*
    if $\|\mathbf{w}_{k+1} - \mathbf{w}_k\| \,/\, \max(1, \|\mathbf{w}_k\|) < \varepsilon_{\text{stop}}$ **then**
        break *(convergence reached)*
    end if
end for
return $\mathbf{w}_{k+1}$

Initialization. We initialize $\mathbf{w}_0$ with the unregularized least-squares solution $\mathbf{A}^{-1}\mathbf{b}$: it is already available (we compute $\mathbf{A}$ and $\mathbf{b}$ once at the start anyway), the relative magnitudes of its components are a reasonable proxy for which weights the L_1 penalty will shrink, and no component is exactly zero, so the weights $(\mathbf{W}_0)_{ii} = |(w_0)_i|^{-1/2}$ are well defined without having to lean on ε_{thr} at the first step.

Stopping criterion. The algorithm terminates when the relative change between successive iterates falls below $\varepsilon_{\text{stop}}$,

$$\frac{\|\mathbf{w}_{k+1} - \mathbf{w}_k\|}{\max(1, \|\mathbf{w}_k\|)} < \varepsilon_{\text{stop}}\,,$$

where the $\max(1, \|\mathbf{w}_k\|)$ normalization makes the test scale-invariant and prevents false triggers when $\|\mathbf{w}_k\|$ is small.

2.5 Convergence analysis

Monotone decrease. As derived in Section 2.2 from the MM framework, IRLS monotonically decreases the objective: $f(\mathbf{w}_{k+1}) \leq f(\mathbf{w}_k)$ for all k. Since f is bounded below by zero, the sequence $\{f(\mathbf{w}_k)\}$ converges.

Theorem 2.2 (Fixed-point consistency)**.** *If the sequence $\{\mathbf{w}_k\}$ generated by IRLS converges to a point $\mathbf{w}^*$ such that $|(w^*)_i| > \varepsilon_{\text{thr}}$ for all i, then $\mathbf{w}^*$ satisfies the optimality conditions of the original problem* (2.1)*.*

Proof. At a fixed point $\mathbf{w}_k = \mathbf{w}_{k+1} = \mathbf{w}^*$, the weighted normal equations (2.9) give:

$$\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + 2\lambda_{\text{IRLS}}\, \mathbf{W}_*^T\mathbf{W}_*\mathbf{w}^* = \mathbf{0}$$

Since $|(w^*)_i| > \varepsilon_{\text{thr}}$ for all i, the threshold in (2.5) is inactive and $(\mathbf{W}_*^T\mathbf{W}_*)_{ii} = |(w^*)_i|^{-1}$ exactly. The i-th component of the regularizer term is then:

$$(\mathbf{W}_*^T\mathbf{W}_*\mathbf{w}^*)_i = \frac{(w^*)_i}{|(w^*)_i|} = \operatorname{sign}((w^*)_i)$$

Substituting and using $\lambda_{\text{IRLS}} = \frac{1}{2}\lambda_{\text{LASSO}}$ from (2.8):

$$\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + \lambda_{\text{LASSO}} \operatorname{sign}(\mathbf{w}^*) = \mathbf{0}$$

Since $|(w^*)_i| \neq 0$ for all i, the L_1 norm is differentiable at $\mathbf{w}^*$, so $\partial\|\mathbf{w}^*\|_1 = \{\operatorname{sign}(\mathbf{w}^*)\}$ [14]. By the sum rule for subdifferentials [20], the equation above is exactly $\mathbf{0} \in \partial f(\mathbf{w}^*)$, the optimality condition of (2.1) [14]. □

Convergence rate. In practice, the IRLS scheme used here exhibits linear (geometric) convergence of the iterates: this is consistent with the analysis carried out by Daubechies et al. [8] for the closely related basis-pursuit IRLS algorithm, where local exponential (linear-rate) convergence is established under sparsity and null-space assumptions on the design matrix. The error $\|\mathbf{w}_k - \mathbf{w}^*\|$ thus shrinks by a constant factor at each iteration, set by the condition number of $\mathbf{Q}_k = \mathbf{X}^T\mathbf{X} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$. The extra diagonal contribution $2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k \succ 0$ behaves like a Tikhonov-style preconditioner, and typically leaves $\mathbf{Q}_k$ better-conditioned than $\mathbf{X}^T\mathbf{X}$ alone, which is what makes the per-iteration linear solve inexpensive and well-conditioned. We do not claim a formal proof of linear convergence for the ELM LASSO setting; the empirical linear rate observed in Section 5.2 is reported as such, with [8] cited only as the closest available analysis under different (basis-pursuit) hypotheses.

Verification of hypotheses for the ELM problem. The two assumptions of Theorem 2.2 are satisfied for (2.1) as follows.

1. *Convergence of the sequence.* We work under the assumption that $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}}\mathbf{W}_1^T)$ has full column rank, which is standard for the ELM setting when $M \geq H$ and $\mathbf{W}_1$ is drawn at random with a nonlinear σ [25].

Under this assumption f is strictly convex (Proposition 1.1) and admits a unique minimizer $\mathbf{w}^*$. The convergence $\mathbf{w}_k \to \mathbf{w}^*$ follows in four steps: *(i)* IRLS monotonically decreases f (Section 2.2) and $f \geq 0$, so $\{f(\mathbf{w}_k)\}$ is non-increasing and bounded below, hence it converges; *(ii)* f is coercive (Proposition 1.1), so every sublevel set is compact and $\{\mathbf{w}_k\}$ has at least one accumulation point $\bar{\mathbf{w}}$; *(iii)* continuity of the surrogate $Q(\cdot, \mathbf{w}_k)$ and the MM descent property $Q(\mathbf{w}_{k+1}, \mathbf{w}_k) \leq Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$ force $\bar{\mathbf{w}}$ to be a fixed point of the IRLS iteration; *(iv)* by Theorem 2.2, every such fixed point satisfying $|(\bar{w})_i| > \varepsilon_{\text{thr}}$ is a global minimizer, so $\bar{\mathbf{w}} = \mathbf{w}^*$ by uniqueness; since every accumulation point coincides with $\mathbf{w}^*$, the full sequence converges.

2. *Non-zero components at convergence.* The hypothesis $|(w^*)_i| > \varepsilon_{\text{thr}}$ holds for the active (non-zero) components of the sparse solution. Components that collapse to zero within ε_{thr} are effectively frozen by the weight clipping (2.5) and drop out of the optimality condition (1.4) — which is exactly the behaviour we want, because these components correspond to features that the L_1 penalty has marked as inactive.

2.6 Computational cost

The per-iteration cost of IRLS breaks down as follows:

- **Weight update $\mathbf{W}_k$**: $O(H)$ — simply iterating over the components of $\mathbf{w}_k$.
- **Matrix update $\mathbf{Q}_k = \mathbf{A} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$**: $O(H)$ — since $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ is pre-computed and only the diagonal changes.
- **Linear system solve $\mathbf{Q}_k\mathbf{w}_{k+1} = \mathbf{b}$**: $O(H^3/3)$ with Cholesky, or $O(pH^2)$ with CG for p CG steps.

The dominant cost per iteration is the linear system solve: $O(H^3)$ with Cholesky. However, the total number of IRLS iterations is typically small (10–50), and the pre-computation of $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ (costing $O(MH^2)$) is amortized over all iterations. The overall cost is therefore:

$$O(MH^2) + k_{\text{IRLS}} \cdot O(H^3)$$

For problems where $H \ll M$ (many training samples, moderate hidden layer size), the pre-computation dominates and IRLS is very efficient.

2.7 Expected behavior on our problem

From the analysis above we expect the following behaviour on the ELM LASSO problem. The objective $f(\mathbf{w}_k)$ decreases at every iteration, with no oscillations, which makes IRLS easy to monitor and to debug: a single non-decreasing step is

a red flag. On a semilog plot of $f(\mathbf{w}_k) - f^*$ against k, we expect an approximately straight line, with a slope set jointly by the conditioning of $\mathbf{Q}_k$ and by the safety threshold ε_{thr}.

Sparsity is produced dynamically rather than by an external thresholding step: components $(w_k)_i$ that start out small get assigned increasingly large weights $|(w_k)_i|^{-1}$, which pushes them further toward zero at the next iteration; the stable outcome is a weight vector with many components inside an ε_{thr}-ball of zero. The quantitative trade-offs are then driven by two parameters we chose early on: the regularization strength λ_{LASSO} (the model parameter), and the safety threshold ε_{thr} (a purely numerical one). Since $\lambda_{\text{IRLS}} = \frac{1}{2}\lambda_{\text{LASSO}}$ is fixed by the majorization, the only free knob in the model is λ_{LASSO}: larger values give sparser solutions but a higher training residual, smaller values do the opposite; conditioning of $\mathbf{Q}_k$ also depends on λ_{LASSO}, so the convergence speed moves with it. For ε_{thr}, smaller values approximate the true L_1 norm more faithfully and yield sharper sparsity, but pay the price in stability whenever a component drifts very close to zero; a value of order 10^{-8} is a reasonable starting point in this trade-off, and we adopt it as our default to be validated experimentally.

Chapter 3

Algorithm 2: Deflected Subgradient Method

In this chapter, we apply the deflected subgradient method with Polyak step and with target level (SGPTL) to the ELM LASSO problem.

3.1 Motivation: why the plain subgradient method is inadequate for ELM LASSO

The ELM LASSO problem

$$f(\mathbf{w}) = \frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda_{\text{LASSO}} \|\mathbf{w}\|_1 , \tag{3.1}$$

is nonsmooth due to the ℓ_1 penalty. While the plain subgradient method guarantees convergence for convex nonsmooth problems, two structural limitations make it impractical for this problem:

1. **Zig-zagging and slow practical convergence:**
   The subgradient update $\mathbf{w}_{i+1} = \mathbf{w}_i - \alpha_i \mathbf{g}_i$ uses only the current subgradient with no memory of previous iterations [21], which on poorly conditioned $\mathbf{X}^T\mathbf{X}$ produces severe oscillations. The complexity lower bound for first-order methods on nonsmooth convex problems is $\Omega(L^2/\varepsilon^2)$ [10, 4], with L a uniform bound on the subgradients. Plain subgradient already attains this optimal rate, but on ELM LASSO the constant L depends on $\|\mathbf{X}\|_2$ and on the radius of the sublevel set containing the iterates, both of which can be large for poorly scaled or ill-conditioned data, so practical convergence is slow.

2. **Stringent stepsize requirements:**
   Convergence requires the Decreasing Stepsize Scheduling condition [21], $\sum_i \alpha_i = \infty$ and $\sum_i \alpha_i^2 < \infty$, which forces the steps to shrink quickly (e.g.

$\alpha_i = c/i$) and stalls practical progress on the accuracy targets we care about.

Deflection addresses both issues by injecting memory into the search direction, and is the variant we adopt for ELM LASSO.

3.1.1 The deflection mechanism

Deflected subgradient methods [13] replace the raw subgradient with a direction that blends the current subgradient with the previous search direction:

$$\mathbf{d}_i = \gamma_i \, \mathbf{g}_i + (1 - \gamma_i) \, \mathbf{d}_{i-1} \quad (i \geq 1), \qquad \mathbf{d}_0 = \mathbf{g}_0, \tag{3.2}$$

where $\gamma_i \in (0, 1]$ is the deflection parameter and the first iteration is handled separately (no previous direction is available). The update becomes $\mathbf{w}_{i+1} = \mathbf{w}_i - \alpha_i \, \mathbf{d}_i$. Setting $\gamma_i = 1$ at every step recovers the plain subgradient method, whereas $\gamma_i < 1$ gives weight to the memory term $\mathbf{d}_{i-1}$, smoothing out the trajectory. This damps the immediate direction reversals that make the plain subgradient method zig-zag on badly conditioned ELM LASSO instances. We pick γ_i greedily to minimize $\|\mathbf{d}_i\|$, which, coupled with the Polyak-style stepsize introduced below, maximizes the reduction achievable at iteration i [17, 13].

3.2 Convergence framework: balancing deflection and stepsize

For deflected methods on ELM LASSO, the deflection γ_i and stepsize α_i must be carefully coordinated. The theoretical framework is in [13]. We use the stepsize-restricted approach [13]:

$$\alpha_i = \frac{\beta_i \, (f(\mathbf{w}_i) - f^*)}{\|\mathbf{d}_i\|_2^2}, \quad \beta_i \leq \gamma_i. \tag{3.3}$$

The condition $\beta_i \leq \gamma_i$ ensures stronger deflection is balanced by smaller steps, preventing large moves in memory-weighted non-descent directions. Since f^* is unknown for ELM LASSO, we replace it with target level $f_{\text{ref}} - \delta$ (Section 3.3), providing an adaptive estimate throughout the run.

Practical handling of the multiplier β_i.
We take $\beta_i = \min(\beta, \gamma_i)$ with $\beta = 1$, the stepsize-restricted rule of [22, Algorithm SGPTL]; this enforces $\beta_i \leq \gamma_i$ by construction, the hypothesis under which the deflected-Polyak rate (used in Theorem 3.1) is established in [13], with original reference [7]. The clip is operational on ELM LASSO only because we pair it with $\gamma_i \geq \gamma_{\min} > 0$ on the deflection parameter, motivated below.

Why $\gamma_{\min} > 0$ is required. Without a floor the closed-form greedy minimiser of eq. (3.4) can be projected onto $\gamma = 0$. This is not a pathological corner case but the typical regime any descent run enters within a few tens of iterations: once $\mathbf{w}_i$ approaches a near-stationary region of f, consecutive subgradients $\mathbf{g}_i, \mathbf{g}_{i-1}$ become well aligned, the parabola coefficients of eq. (3.7) satisfy $\|\mathbf{d}_{i-1}\|_2^2 - \langle \mathbf{g}_i, \mathbf{d}_{i-1} \rangle \to 0^+$ from below the denominator $\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2$, and the unconstrained minimiser falls below 0 — it gets clipped to the left boundary. The deflected direction collapses to $\mathbf{d}_i = \mathbf{d}_{i-1}$ (pure memory), $\gamma_i = 0$, and through the clip $\beta_i = 0$ and the Polyak step $\alpha_i = 0$. The iterate freezes.

We therefore project the greedy minimiser of eq. (3.4) onto $[\gamma_{\min}, 1]$ instead of $[0, 1]$, with $\gamma_{\min} = 0.05$ as the default. The choice of $\gamma_{\min}$ trades two competing concerns. On one hand, $\gamma_{\min}$ should be small enough that the floor rarely binds in the pre-stationary phase, where the unconstrained minimiser already lies inside $[0, 1]$: this preserves the $\|\mathbf{d}_i\|$-minimisation property that justifies the greedy choice. On the other hand, $\gamma_{\min}$ should be large enough that $\beta_i = \min(\beta, \gamma_i) \geq \gamma_{\min}$ keeps the Polyak step α_i bounded away from zero and the iterate moving. We sweep $\gamma_{\min} \in \{0.05, 0.1, 0.2, 0.5\}$ on the convergence instance of Section 5.2 and find final record gaps in the $\sim 10^{-7}$–10^{-5} range across the entire grid, with $\gamma_{\min} = 0.05$ marginally best ($\bar{f}^i - f^* \approx 5 \cdot 10^{-8}$ at $\rho = 0.7$); the algorithm is robust to the floor inside this range and we adopt $\gamma_{\min} = 0.05$ in the rest of the report.

Why the floor is essential, not parametric. A natural objection is that the freeze in the unfloored case ($\gamma_{\min} = 0$) might be a tuning issue resolved by a different (δ_0, ρ) choice. Section 5.1 of Chapter 5 reports an empirical sweep across 25 combinations of $\delta_0 \in \{0.01, \ldots, 1.0\} f^*$ and $\rho \in \{0.5, \ldots, 0.99\}$ with $\gamma_{\min} = 0$ active and the literal clip in place: only one configuration ($\delta_0 = 1.0\, f^*$, $\rho = 0.99$) reaches a record gap below 10^{-3}, and even there the gap stalls two orders of magnitude above what the floored variant achieves at the same parameter point. The floor is therefore not a parameter fine-tune but a structural ingredient required for the greedy-γ deflection scheme to be operational on this problem class — a fact we attribute to ELM LASSO's smooth-like behaviour near the optimum, where consecutive subgradients align and the parabola of eq. (3.7) pushes the unconstrained minimiser below zero.

3.2.1 Optimal deflection parameter

At each iteration on ELM LASSO, we choose γ_i to minimize the deflected direction norm subject to a strictly positive lower bound $\gamma_{\min}$:

$$\gamma_i \in \underset{\gamma \in [\gamma_{\min}, 1]}{\arg\min} \|\gamma\, \mathbf{g}_i + (1 - \gamma)\, \mathbf{d}_{i-1}\|_2^2 . \tag{3.4}$$

Since the Polyak stepsize has denominator $\|\mathbf{d}_i\|^2$, minimizing this maximizes step length and progress. The constraint $\gamma_i \geq \gamma_{\min}$ keeps the stepsize-restricted condition $\beta_i = \min(\beta, \gamma_i) \geq \gamma_{\min}$ operational (Section 3.2); we use $\gamma_{\min} = 0.05$

throughout and discuss the rationale in that section. The problem is quadratic in γ with closed-form solution [13]:

$$\gamma^* = \frac{\|\mathbf{d}_{i-1}\|_2^2 - \langle \mathbf{g}_i, \mathbf{d}_{i-1}\rangle}{\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2}, \quad \gamma_i = \min(1, \max(\gamma_{\min}, \gamma^*)). \tag{3.5}$$

Derivation of γ^*.
We solve the unconstrained version of (3.4) over $\gamma \in \mathbb{R}$ and then project onto $[0, 1]$. Define $h(\gamma) := \|\gamma\, \mathbf{g}_i + (1-\gamma)\, \mathbf{d}_{i-1}\|^2$. Expanding by bilinearity of the inner product:

$$\begin{aligned} h(\gamma) &= \gamma^2 \|\mathbf{g}_i\|_2^2 + 2\gamma(1-\gamma)\langle \mathbf{g}_i, \mathbf{d}_{i-1}\rangle + (1-\gamma)^2 \|\mathbf{d}_{i-1}\|_2^2 \\ &= \gamma^2\big(\|\mathbf{g}_i\|_2^2 - 2\langle \mathbf{g}_i, \mathbf{d}_{i-1}\rangle + \|\mathbf{d}_{i-1}\|_2^2\big) + 2\gamma\big(\langle \mathbf{g}_i, \mathbf{d}_{i-1}\rangle - \|\mathbf{d}_{i-1}\|_2^2\big) + \|\mathbf{d}_{i-1}\|_2^2 . \end{aligned} \tag{3.6}$$

Recognising the first factor as $\|\mathbf{g}_i - \mathbf{d}_{i-1}\|^2$:

$$h(\gamma) = \|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2\, \gamma^2 + 2\big(\langle \mathbf{g}_i, \mathbf{d}_{i-1}\rangle - \|\mathbf{d}_{i-1}\|_2^2\big)\gamma + \|\mathbf{d}_{i-1}\|_2^2 . \tag{3.7}$$

Case $\mathbf{g}_i = \mathbf{d}_{i-1}$: the leading coefficient $\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2 = 0$, so $h(\gamma) = \|\mathbf{d}_{i-1}\|_2^2$ is constant on $[0, 1]$; moreover, $\mathbf{d}_i = \gamma\mathbf{g}_i + (1-\gamma)\mathbf{d}_{i-1} = \mathbf{d}_{i-1}$ for all γ, so the deflected direction is identical regardless of the choice; we set $\gamma_i = 1$ by convention.

Case $\mathbf{g}_i \neq \mathbf{d}_{i-1}$: h is a strictly convex parabola in γ (positive leading coefficient). Its unique unconstrained minimizer satisfies $h'(\gamma^*) = 0$:

$$h'(\gamma) = 2\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2\, \gamma + 2\big(\langle \mathbf{g}_i, \mathbf{d}_{i-1}\rangle - \|\mathbf{d}_{i-1}\|_2^2\big) = 0 \implies \gamma^* = \frac{\|\mathbf{d}_{i-1}\|_2^2 - \langle \mathbf{g}_i, \mathbf{d}_{i-1}\rangle}{\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2},$$

which is exactly (3.5). The constrained minimum over $[\gamma_{\min}, 1]$ is $\gamma_i = \min(1, \max(\gamma_{\min}, \gamma^*))$: since the parabola opens upward, if $\gamma^* < \gamma_{\min}$ then h is increasing on $[\gamma_{\min}, 1]$ and the minimum is attained at $\gamma = \gamma_{\min}$; if $\gamma^* > 1$ then h is decreasing on the interval and the minimum is at $\gamma = 1$; if $\gamma^* \in [\gamma_{\min}, 1]$ the interior solution applies.

Interpretation.
The denominator $\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2$ measures the squared distance between the current subgradient and the previous direction: the larger it is, the more γ^* shrinks and the more the deflected direction borrows from memory $\mathbf{d}_{i-1}$. The numerator $\|\mathbf{d}_{i-1}\|_2^2 - \langle \mathbf{g}_i, \mathbf{d}_{i-1}\rangle$ captures alignment: it is small (and γ^* small) when $\mathbf{g}_i$ and $\mathbf{d}_{i-1}$ are well aligned, and large when they oppose each other. In the zig-zag scenario $\mathbf{g}_i \approx -\mathbf{d}_{i-1}$ with comparable magnitudes $\|\mathbf{g}_i\|_2 \approx \|\mathbf{d}_{i-1}\|_2$, the numerator is approximately $\|\mathbf{d}_{i-1}\|_2^2 + \|\mathbf{d}_{i-1}\|_2^2 = 2\|\mathbf{d}_{i-1}\|_2^2$ while the denominator is approximately $\|2\mathbf{d}_{i-1}\|_2^2 = 4\|\mathbf{d}_{i-1}\|_2^2$, giving $\gamma^* \approx 1/2$: $\mathbf{d}_i$ splits evenly between the two conflicting directions, halving the oscillation.

3.3 The target-level mechanism

The Polyak stepsize [17] requires knowledge of f^*. The target-level mechanism circumvents this by maintaining an adaptive estimate throughout the run.

The fundamental relationship for subgradient methods [21] is:

$$\|\mathbf{w}_{i+1} - \mathbf{w}^*\|_2^2 \leq \|\mathbf{w}_i - \mathbf{w}^*\|_2^2 - 2\alpha_i(f(\mathbf{w}_i) - f^*) + (\alpha_i)^2 \|\mathbf{g}_i\|_2^2 . \tag{3.8}$$

The ideal stepsize $\alpha_i^* = (f(\mathbf{w}_i) - f^*)/\|\mathbf{g}_i\|_2^2$ [17] maximizes guaranteed reduction but requires f^*.

For ELM LASSO we replace f^* with the target level $f_{\text{ref}} - \delta$ [22], where f_{ref} is the best function value found so far and $\delta > 0$ is the expected improvement; this is the same expression used in the Polyak-step numerator of Algorithm 2. When the target is accurate (i.e. $f_{\text{ref}} - \delta \approx f^*$) the algorithm tracks the ideal Polyak step and achieves substantial progress; when the target is too optimistic (i.e. $f_{\text{ref}} - \delta < f^*$), the numerator $f(\mathbf{w}_i) - (f_{\text{ref}} - \delta) = (f(\mathbf{w}_i) - f^*) + (f^* - f_{\text{ref}} + \delta)$ *overestimates* $f(\mathbf{w}_i) - f^*$ and the resulting steps are larger than the ideal Polyak ones; the real failure mode is that the improvement condition $f(\mathbf{w}_{i+1}) \leq f_{\text{ref}} - \delta/2$ used by Algorithm 2 can never be satisfied (since $f(\mathbf{w}_{i+1}) \geq f^* > f_{\text{ref}} - \delta > f_{\text{ref}} - \delta/2$), so no progress is registered and the iterates stagnate. We detect stalling by tracking the accumulated displacement $r = \sum \alpha_i \|\mathbf{d}_i\|$: if $r > R$ (the patience threshold) without significant improvement, we contract $\delta \leftarrow \delta \cdot \rho$ with $\rho < 1$ and retry. Repeated contractions drive $\delta \to 0$, so $f_{\text{ref}} - \delta \to f_{\text{ref}} \geq f^*$ and the target is restored above f^*.

3.4 The complete algorithm

Algorithm 2 presents the full Deflected Subgradient Method with target level for ELM LASSO.

Correspondence with the theory.
Algorithm 2 implements one-to-one the SGPTL rule of slide 14 of the course (Lessons Optimization) and Lemma 3.8 of [7]. Lines 12–13 are the stepsize-restricted Polyak step ($\beta_i \leq \gamma_i$, hypothesis of Theorem 3.1); lines 16–21 are the three-branch update of $(f_{\text{ref}}, \delta, r)$ exactly as in Lemma 3.8. The single deviation from the slide's pseudocode is the clip on γ_i at line 9: we project the greedy minimiser of (3.4) onto $[\gamma_{\min}, 1]$ rather than $[0, 1]$, with $\gamma_{\min} = 0.05$. This is not a workaround but a structural choice that realises condition (3.5) of [7] with $\beta^* = \gamma_{\min} > 0$: without the floor, the greedy choice (3.5) collapses to 0 on near-aligned consecutive subgradients, $\beta_i = \min(\beta, \gamma_i) \to 0$ violates (3.5), and the convergence hypothesis of Theorem 3.1 fails. The floor is the minimal modification that keeps the theory applicable; we discuss the choice $\gamma_{\min} = 0.05$ in Section 3.2.

Initial-iteration convention.
The first iteration $i = 0$ is treated separately because (3.5) requires a previous

Algorithm 2 Deflected Subgradient with Target Level (SGPTL) for ELM LASSO [13]

Require: f (ELM LASSO objective); $i_{\max}$; $\beta \in (0,1]$; $\delta_0 > 0$; $R > 0$; $\rho \in (0,1)$; $\gamma_{\min} \in (0,1]$

$\mathbf{w}_0 \leftarrow (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$ *(same OLS warm start as IRLS)*
$r \leftarrow 0$; $\delta \leftarrow \delta_0$; $f_{\text{ref}} \leftarrow \bar{f} \leftarrow f(\mathbf{w}_0)$
for $i = 0, 1, \ldots, i_{\max}$ **do**
    Compute $\mathbf{g} \in \partial f(\mathbf{w}_i)$ *(subgradient of LASSO)*
    if $i = 0$ **then** *(no $\mathbf{d}_{-1}$ available)*
        $\gamma_i \leftarrow 1$; $\mathbf{d}_i \leftarrow \mathbf{g}$
    else
        Compute γ_i via (3.5) *(optimal deflection, clipped to $[\gamma_{\min}, 1]$)*
        $\mathbf{d}_i \leftarrow \gamma_i\,\mathbf{g} + (1-\gamma_i)\,\mathbf{d}_{i-1}$ *(deflected direction)*
    end if
    $\beta_i \leftarrow \min(\beta, \gamma_i)$ *(stepsize-restricted, $\beta = 1$ default; see §3.2)*
    $\alpha_i \leftarrow \beta_i\,(f(\mathbf{w}_i) - (f_{\text{ref}} - \delta)) \,/\, \|\mathbf{d}_i\|_2^2$ *(Polyak step)*
    $\mathbf{w}_{i+1} \leftarrow \mathbf{w}_i - \alpha_i\,\mathbf{d}_i$ *(update)*
    $\bar{f} \leftarrow \min(\bar{f},\ f(\mathbf{w}_{i+1}))$ *(update record)*
    if $f(\mathbf{w}_{i+1}) \le f_{\text{ref}} - \delta/2$ **then** *(sufficient descent)*
        $f_{\text{ref}} \leftarrow \bar{f}$; $r \leftarrow 0$
    else if $r > R$ **then** *(travel-distance patience exhausted)*
        $\delta \leftarrow \delta\,\rho$; $r \leftarrow 0$
    else
        $r \leftarrow r + \alpha_i\,\|\mathbf{d}_i\|_2$
    end if
end for
return $\mathbf{w}_{\text{best}}$ (iterate achieving $\bar{f}$)

direction $\mathbf{d}_{-1}$ that does not exist yet; setting $\gamma_0 = 1$ realises the convention $\mathbf{d}_{-1} = \mathbf{0}$ in eq. (3.2), so $\mathbf{d}_0 = \mathbf{g}_0$. Plugging $\mathbf{d}_{-1} = \mathbf{0}$ directly into (3.5) would yield 0/0; the convention $\gamma_0 = 1$ is the same one used in the slide.

Numerical safeguards.
Three pure floating-point safeguards complete the implementation; none changes the algorithmic behaviour, none reacts to algorithmic state (no δ contraction, no $\mathbf{d}$ reset), and none is in the pseudocode of Algorithm 2. *(i)* If $\|\mathbf{d}_i\|^2 < 10^{-30}$ we exit the loop early: the deflected direction is numerically indistinguishable from zero, so α_i would divide by zero. This is a termination, not an alteration. *(ii)* If $f(\mathbf{w}_i) - (f_{\text{ref}} - \delta) \leq 0$, the Polyak numerator is non-positive and the step would point backwards. We set $\alpha_i = 0$ as prescribed by condition (3.5) of [7], and trigger the sufficient-descent update $f_{\text{ref}} \leftarrow \bar{f}$, $r \leftarrow 0$ on $\mathbf{w}_i$ itself (since $f(\mathbf{w}_i) \leq f_{\text{ref}} - \delta \leq f_{\text{ref}} - \delta/2$): this is the theory-aligned response, not a δ contraction. *(iii)* If $\mathbf{w}_{i+1}$ contains a non-finite component (NaN/Inf), which can happen when $\|\mathbf{d}_i\|^2$ is tiny but above the cutoff of (i) and α_i overflows, we skip the iteration and proceed. None of δ, f_{ref}, r is touched.

A note on what is *not* in the algorithm.
We deliberately do not include any non-theoretical safeguard such as an iteration-count fallback for δ, an overshoot rescue snapping $\mathbf{w}$ to $\mathbf{w}_{\text{best}}$ on large f-jumps, or extra contractions of δ outside line 19. Such mechanisms can rescue specific failure modes (the greedy collapse $\gamma_i \to \gamma_{\min}$ stalls the algorithm on ELM LASSO well before the record converges to f^*, as we document in Chapter 5) but they have no support in the theory of [7] and would break the proof of Theorem 3.1; we prefer to expose the algorithm's empirical limitations honestly rather than mask them.

3.4.1 Parameter calibration for ELM LASSO

SGPTL on ELM LASSO has five parameters: the step modulation β, the deflection floor $\gamma_{\min}$, the initial target gap δ_0, the contraction factor ρ, and the patience threshold R. We discuss them in order from the most theoretically constrained to the most empirically calibrated. The point worth stressing up front is that f^* — the optimum that the algorithm is trying to reach — is *not* an admissible input to any of these choices: tuning the optimiser against the quantity it is searching for is the optimisation analogue of training a model on its test set. All hyperparameters below are set from f^*-free quantities.

$\beta = 1$ **(step modulation).** [7, (3.1), p. 364] admits any $\beta_k \in (0, 1]$ in the corrected Polyak stepsize. Slide 12 of the course ([17]) shows that, under the classical Polyak step $\alpha_k = \beta\,(f(\mathbf{w}_k) - f^*)/\,\|\mathbf{g}_k\|^2$, the value $\beta = 1$ is exactly the minimiser of the one-step distance bound $\|\mathbf{w}_{k+1} - \mathbf{w}^*\|^2$, i.e. the maximum-progress choice. The deflected variant in slide 15 inherits the same maximum-progress property, with the deflection coefficient γ_i providing the dynamic damping; we

therefore fix $\beta = 1$ and never sweep it. The effective per-iteration multiplier $\beta_i = \min(\beta, \gamma_i)$ in Algorithm 2 is then driven entirely by γ_i and by the floor $\gamma_{\min}$.

$\gamma_{\min} = 0.05$ **(deflection floor).** Condition (3.5) of [7, p. 365] is a *strictly necessary* hypothesis for the convergence of stepsize-restricted deflected methods: it requires $\beta_k \geq \beta^* > 0$ uniformly whenever $\lambda_k \geq 0$. With our $\beta_i = \min(1, \gamma_i)$, the uniform bound β^* is realised exactly when $\gamma_i \geq \gamma_{\min} > 0$, in which case we may take $\beta^* = \gamma_{\min}$. The greedy choice (3.5) of γ_i violates (3.5) on ELM LASSO (the closed-form minimiser collapses to 0 on near-aligned consecutive subgradients, see §3.2), so the floor is the minimal modification that keeps the convergence theorem applicable. The value $\gamma_{\min} = 0.05$ is empirically calibrated: at this floor the clip activates only in degenerate steps (the unconstrained greedy γ lies in $[\gamma_{\min}, 1]$ on a typical step), while the convergence rate $1/(\gamma_{\min}\sqrt{k})$ of Theorem 3.1 inflates by only $20\times$ relative to the unfloored Polyak rate. The sweep across $\gamma_{\min} \in \{0.05, 0.1, 0.2, 0.5\}$ in Chapter 5 produces gaps consistently in $[10^{-7}, 10^{-5}]$, with $\gamma_{\min} = 0.05$ marginally best.

δ_0 **(initial target gap).** The theory of [7, Lemma 3.8] requires only $\delta_0 \in (0, \infty)$; the convergence proof does not depend on the specific value, only on the contraction rule that drives $\delta_k \to 0$. The course slide 14 explicitly flags $\delta_0 > 0$ as "`??`" — a free parameter. We therefore set $\delta_0 = c\, f(\mathbf{w}_0)$ where $\mathbf{w}_0$ is the OLS warm start (an $O(MH^2)$ pre-computation that does not see f^*) and $c = 0.1$. The scale $f(\mathbf{w}_0)$ is an upper bound on f^* (since adding the regulariser strictly increases the value at a non-LASSO minimiser), and is the natural problem-dependent quantity for an order-of-magnitude estimate of the initial gap. The constant $c = 0.1$ is calibrated empirically in Section 5.3; the sweep across $c \in \{0.01, 0.05, 0.1, 0.5, 1\}$ produces gaps within a single order of magnitude, so the choice is not delicate. Section 5.3 also compares this f^*-free choice against the diagnostic oracle $\delta_0 = 0.1\, f^*$ (reported for theoretical understanding only, never used in actual runs); the two are numerically indistinguishable on the tested instances.

$\rho = 0.7$ **(threshold contraction).** [7, Lemma 3.8] requires only $\mu \in (0, 1)$; the course slide 14 echoes the same open-interval constraint. No specific value is prescribed in any of the references we consulted ([3, 23, 1]). Empirically we sweep $\rho \in \{0.3, 0.5, 0.7, 0.8, 0.9, 0.95\}$ in Section 5.3 on the synthetic instance and on the two real-data ELM matrices; $\rho = 0.7$ is the value at most a factor-of-six away from the best on every tested instance, and is solidly inside the moderate range $[0.5, 0.8]$ where the target $f_{\text{ref}} - \delta$ tracks f^* at the right pace.

$R = 1$ **(travel-distance patience).** [7, Lemma 3.8] requires only $R > 0$; slide 14 flags it as "`???`", a free parameter not prescribed in any of [3, 23]. The value R controls the cadence of δ-contractions: an R much larger than the typical per-iteration step length $\alpha_i \|\mathbf{d}_i\|$ leaves the $r_k > R$ branch inactive (the

algorithm relies on the rare sufficient-descent firings to update f^k_{ref}), an R much smaller wastes the patience window. On ELM LASSO with $\gamma_{\min} = 0.05$ and the OLS warm start, $\alpha_i \|\mathbf{d}_i\|$ is of order 10^{-3}, so we pick $R = 1$, which fires the contraction roughly every 10^3 stalled iterations. An earlier draft of this report used $R = 10\sqrt{i_{\max}} \approx 894$ at $i_{\max} = 8000$, three orders of magnitude above the natural scale, with the consequence that the $r > R$ branch never fired and the algorithm appeared to stall (see Section 5.1.1 for the diagnosis). The choice $R = 1$ is theory-aligned — only $R > 0$ is required — and is the single calibration choice that the first submission got wrong.

3.4.2 Starting point and patience-threshold calibration

Algorithm 2 uses the same OLS warm start as IRLS, $\mathbf{w}_0 = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$, so that both methods minimise the same problem from the same point and the empirical comparison is on equal footing. The travel-distance patience threshold is set to $R = 1$, a small absolute value chosen so the threshold is comparable to the per-iteration step length scale $\alpha_i \|\mathbf{d}_i\| \sim 10^{-3}$ on ELM LASSO with the OLS warm start and $\gamma_{\min} = 0.05$. The travel-distance branch then fires $\mathcal{O}(10)$–$\mathcal{O}(100)$ times per 8 000-iteration run, depending on ρ, contracting δ at a sustained pace as the theory expects (Section 5.3 reports the rate). The choice $R = 1$ is within the theoretical framework of [7] (only $R > 0$ is required by Lemma 3.8) and is the only calibration choice that we make outside the slide's pseudocode; the rest of the parameters follow the slide directly.

Implementation history and rationale. The first submission of this project used additional mechanisms on top of Algorithm 2: (a) an iteration-count δ-contraction trigger (auxiliary counter c tracking iterations without improvement of $\bar{f}$, with the rule "if $c \geq R_{\text{iter}}$ then $\delta \leftarrow \rho\delta$ and $\mathbf{d}_{i-1} \leftarrow \mathbf{0}$"); (b) an overshoot rescue that snapped $\mathbf{w}$ back to $\mathbf{w}_{\text{best}}$ on large Polyak-step blow-ups and contracted δ; and (c) the default $R = 10\sqrt{i_{\max}} \approx 894$ for the travel-distance threshold, which on ELM LASSO was orders of magnitude larger than any realistic accumulated travel between resets, so the $r > R$ branch effectively never fired and the iteration-count fallback was the sole driver of δ-contractions. The instructor's feedback flagged this construction as inconsistent with the theory of [7]: the proof of $\sum_k s_k = \infty$ in Lemma 3.8 requires every δ-contraction to be preceded by at least R of accumulated travel via the $r_k > R$ trigger, which the iteration-count fallback breaks. Theorem 3.1, which cites [7, Theorem 3.7] applied to Lemma 3.8 verbatim, was therefore not fully applicable to the algorithm we were actually running.

The diagnosis after the feedback was twofold. First, mechanisms (a) and (b) are workarounds that hide bugs rather than fix them, and the proof technique of [7, Lemma 3.8] does not extend to them; they should not be in the algorithm. Second, mechanism (c) — a single integer in our defaults — was the actual reason the theory-pure algorithm appeared to "not work" in the first submission: with R chosen at the right scale, the $r > R$ branch of Lemma 3.8 fires the way the theory needs it to, δ contracts at a sustained pace, and the algorithm

converges within the theoretical envelope of Theorem 3.1. The starting point, on the other hand, turns out to be approximately neutral: with $R = 1$ both the OLS warm start and a cold start $\mathbf{w}_0 = \mathbf{0}$ produce sublinear descent, with the warm start slightly behind on some instances and slightly ahead on others (Section 5.1.1 documents the side-by-side comparison). The second submission therefore: removes (a) and (b); sets the default to $R = 1$; keeps the OLS warm start for both algorithms to retain the apples-to-apples comparison. The empirical cost of switching to the theory-pure rule is moderate (gap $\sim 10^{-5}$ at 8 000 iterations on the convergence instance, instead of the $\sim 10^{-8}$ that the workarounds produced); we view this as the honest baseline of the algorithm and discuss the gap to IRLS in the results and conclusions chapters.

3.5 Application to the ELM LASSO problem

3.5.1 Subgradient computation for ELM LASSO

For the ELM LASSO objective, decompose $f = g+h$ where $g(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$ (smooth) and $h(\mathbf{w}) = \lambda_{\text{LASSO}}\|\mathbf{w}\|_1$ (nonsmooth).

By the sum rule for subdifferentials [20]:

$$\partial f(\mathbf{w}) = \nabla g(\mathbf{w}) + \lambda_{\text{LASSO}}\,\partial\|\mathbf{w}\|_1,$$

where $\nabla g(\mathbf{w}) = \mathbf{X}^T(\mathbf{X}\mathbf{w}-\mathbf{y})$ and the subdifferential of the ℓ_1 norm is componentwise (Section 1.5.3). A subgradient of f at $\mathbf{w}$ is:

$$\mathbf{g} = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda_{\text{LASSO}}\,\mathbf{s}, \quad s_i \in \partial|w_i|. \tag{3.9}$$

At a component where $w_i \neq 0$ the subdifferential is a singleton and $s_i = \text{sign}(w_i)$ is forced; at $w_i = 0$ the subdifferential is the whole interval $[-1, +1]$ and we must pick a value. Writing $\mathbf{q} = \mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$ for the smooth gradient, the minimum-norm element of ∂f at an index with $w_i = 0$ is obtained by minimising $(q_i + \lambda_{\text{LASSO}}\,s_i)^2$ over $s_i \in [-1, +1]$, which gives $s_i^\star = -q_i/\lambda_{\text{LASSO}}$ when $|q_i| \leq \lambda_{\text{LASSO}}$ (so that $g_i = 0$) and $s_i^\star = -\,\text{sign}(q_i)$ otherwise.

Our implementation takes $s_i = 0$ regardless, via `numpy.sign(0) = 0`. This value is always admissible — it lies in $[-1, +1]$ — so $\mathbf{g}$ is a valid subgradient and the convergence guarantee of Theorem 3.1 applies as written. It is *not* the minimum-norm element of ∂f at $\mathbf{w}$: the unconstrained minimum of $(q_i + \lambda_{\text{LASSO}}s_i)^2$ over s_i is $s_i^\star = -q_i/\lambda_{\text{LASSO}}$, which yields $g_i^\star = 0$ when $|q_i| \leq \lambda_{\text{LASSO}}$ and $s_i^\star = -\,\text{sign}(q_i)$ with $|g_i^\star| = |q_i| - \lambda_{\text{LASSO}}$ otherwise; our choice $s_i = 0$ matches neither (except trivially when $q_i = 0$). The discrepancy in $|g_i|$ is at most λ_{LASSO} per affected component, so it inflates the constant L in the convergence rate but not the $O(L^2/\varepsilon^2)$ order. In practice the issue is essentially theoretical for SGPTL on the OLS warm start: the iterate w_i equals zero exactly only at indices that were zero in $\mathbf{w}_0$, an event of zero Lebesgue measure under the Polyak-step update once the algorithm leaves the initial point.

3.5.2 Convergence analysis and hypothesis verification for ELM LASSO

Theorem 3.1 (Convergence of SGPTL, adapted from [22, 13, 7])**.** *Let $f : \mathbb{R}^H \to \mathbb{R}$ be convex with $f^* = \min_{\mathbf{w}} f(\mathbf{w}) > -\infty$ attained at some $\mathbf{w}^*$, and let $\{\mathbf{w}_i\}$ be the sequence generated by Algorithm 2 (the target-level deflected subgradient method we actually run). Assume the subgradients encountered along the run are uniformly bounded, $\|\mathbf{g}_i\|_2 \le L$, and the stepsize-restricted rule $\beta_i \le \gamma_i$ holds with $\gamma_i \ge \gamma_{\min} > 0$. Then the record values $\bar{f}_i = \min_{h \le i} f(\mathbf{w}_h)$ converge to f^*, and once the target-level estimate has sharpened to f^* (i.e. $\delta_k \to 0$ and $f^k_{\mathrm{ref}} \to f^*$, which Lemma 3.8 of [7] guarantees) the asymptotic rate matches the classical Polyak bound [16, 1] inflated by the deflection floor:*

$$\bar{f}_k - f^* \;\le\; \frac{L\,\|\mathbf{w}_0 - \mathbf{w}^*\|_2}{\gamma_{\min}\sqrt{k}}\,, \tag{3.10}$$

so the worst-case complexity to attain $\bar{f}_k - f^* \le \varepsilon$ is $O(L^2/(\gamma_{\min}^2 \varepsilon^2))$.

Proof. We verify the hypotheses of [7, Theorem 3.7] on Algorithm 2 and read off the conclusions. The notation in [7] maps to ours via $\sigma_k \leftrightarrow 0$ (exact oracle), $\alpha_k \leftrightarrow \gamma_i$, $\beta_k \leftrightarrow \beta_i$, $\nu_k \leftrightarrow \alpha_i$, $\mu \leftrightarrow \rho$, $X = \mathbb{R}^H$ (no constraint), and $\lambda_k \leftrightarrow f(\mathbf{w}_i) - (f^k_{\mathrm{ref}} - \delta_k)$.

(a) Deflection-consistency condition (2.13). In [7] condition (2.13) of Lemma 2.4 reads $d_k = \tilde{d}_k \Rightarrow v_{k+1} = \tilde{d}_k$, where $\tilde{d}_k$ is the unprojected deflected direction and $\hat{d}_k = -P_{T_k}(-\tilde{d}_k)$ the projected one. This is the hypothesis under which the "crucial" inequality (2.12) holds, which the stepsize-restricted analysis relies on. With $X = \mathbb{R}^H$ the tangent cone is $T_k = \mathbb{R}^H$ everywhere, so $\hat{d}_k = \tilde{d}_k$ and necessarily $d_k = \tilde{d}_k$ at every iteration; the algorithm sets v_{k+1} to $d_k = \tilde{d}_k$ (Algorithm 2 carries no projection step), so (2.13) is satisfied trivially.

(b) Stepsize condition (3.5): $\lambda_k \ge 0 \Rightarrow \alpha_k \ge \beta_k \ge \beta^* > 0$, $\lambda_k < 0 \Rightarrow \alpha_k = 0$. When $\lambda_k > 0$, line 12 of Algorithm 2 sets $\beta_i = \min(1, \gamma_i) \ge \gamma_{\min}$ via the floor on γ_i, so we may take $\beta^* = \gamma_{\min}$; the inequality $\alpha_k \ge \beta_k$ in (3.5) is $\gamma_i \ge \beta_i$, which is exactly the stepsize-restricted rule. When $\lambda_k \le 0$ the numerical safeguard of Section 3.4 skips the step — equivalent to $\alpha_i = 0$ — and triggers the sufficient-descent update $f^k_{\mathrm{ref}} \leftarrow \bar{f}$, $r \leftarrow 0$ on $\mathbf{w}_i$ itself. The skip realises (3.5) verbatim; the additional f^k_{ref} update is theory-aligned because $\lambda_k \le 0$ means $f(\mathbf{w}_i) \le f^k_{\mathrm{ref}} - \delta_k \le f^k_{\mathrm{ref}} - \delta_k/2$, which is the sufficient-descent condition of [7, Lemma 3.8] evaluated at $\mathbf{w}_i$ rather than at $\mathbf{w}_{i+1}$.

(c) Target-level threshold condition (3.26): requires *either* $f^\infty_{\mathrm{ref}} = f^* = -\infty$ *or* $\liminf_k \delta_k = 0$ together with the series condition (3.25), $\sum_k \lambda_k / \|\mathbf{d}_k\|^2 = \infty$ (equivalent to $\sum_k s_k = \infty$ via (3.28) of [7], where $s_k = \|\mathbf{w}_{k+1} - \mathbf{w}_k\| = \beta_k \lambda_k / \|\mathbf{d}_k\|$). Since on ELM LASSO $f^* > -\infty$, the first branch is excluded and we need the second. [7, Lemma 3.8] provides a concrete update strategy for $(f^k_{\mathrm{ref}}, \delta_k)$ — the sufficient-descent test on $f_{k+1} \le f^k_{\mathrm{ref}} - \delta_k/2$, the target-infeasibility test on $r_k > R$, and the accumulate-r counter — which attains both $\delta_k \to 0$ and $\sum_k s_k = \infty$. Lines 16–21 of Algorithm 2 implement that strategy verbatim under the identification $\mu \leftrightarrow \rho$, so condition (3.26) holds.

Subgradients uniformly bounded. The convex objective f of ELM LASSO is coercive and continuous; the iterates $\{\mathbf{w}_i\}$ remain in a compact sublevel set S_{c_0} for some $c_0 \geq f(\mathbf{w}_0)$ (verification below), and ∂f is uniformly bounded on S_{c_0} by Lipschitz continuity of convex functions on compact sets [15].

Conditions (2.13), (3.5), (3.26) being verified with $\sigma^* = 0$, [7, Theorem 3.7] gives $f_{\text{ref}}^{\infty} \leq f^* + \sigma^* = f^*$, so $\bar{f}_i \to f^*$. The asymptotic rate (3.10) is then the classical Polyak rate [1, Theorem 7.17], [16] applied to the regime where the target-level estimate has sharpened to f^* (Lemma 3.8 of [7] guarantees $\delta_k \to 0$, so eventually the step reduces to the standard Polyak step with f^*); the factor $1/\gamma_{\min}$ is the explicit price of the deflection floor, which contracts the effective stepsize by $\beta_i = \min(1, \gamma_i) \geq \gamma_{\min}$ at every iteration. On ELM LASSO with $\gamma_{\min} = 0.05$ this inflates the iteration count to reach a given ε by $400\times$ relative to the unfloored Polyak rate, but the $O(L^2/\varepsilon^2)$ order is preserved. □

Verification of theorem assumptions for ELM LASSO.

1. **Convexity of** $f(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda_{\text{LASSO}}\|\mathbf{w}\|_1$.

   The smooth part g has Hessian $\nabla^2 g = \mathbf{X}^T\mathbf{X} \succeq 0$ (positive semidefinite), so g is convex. The ℓ_1 norm is convex. The sum of convex functions is convex [12], so f is convex.

   We assume $\mathbf{X}$ has full column rank, as commonly observed in ELM practice when $M \geq H$ [25]; under this assumption $\nabla^2 g \succ 0$, so g is *μ-strongly convex* with modulus $\mu = \lambda_{\min}(\mathbf{X}^T\mathbf{X}) > 0$, and adding the convex term h preserves the property [19, 12], so $f = g + h$ is μ-strongly convex as well. In principle this structure could be exploited by methods designed for strongly-convex nonsmooth problems, which improve the worst-case complexity to $O(L^2/(\mu\varepsilon))$ [19]. However, Algorithm 2 does not incorporate any such mechanism: deflection only dampens oscillations, it does not introduce the kind of acceleration or proximal step needed to exploit μ, so the worst-case guarantee for our SGPTL on ELM LASSO remains the standard $O(L^2/\varepsilon^2)$.

2. **Bounded subgradients for ELM LASSO.**

   The ℓ_1 component has $\|\lambda_{\text{LASSO}}\mathbf{s}\|_2 \leq \lambda_{\text{LASSO}}\sqrt{H}$ (since each $|s_i| \leq 1$). The smooth component $\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$ grows unboundedly as $\|\mathbf{w}\| \to \infty$, so f is not globally Lipschitz on $\mathbb{R}^H$.

   However, f is coercive ($f(\mathbf{w}) \to \infty$ as $\|\mathbf{w}\| \to \infty$), so every sublevel set $S_c = \{\mathbf{w} : f(\mathbf{w}) \leq c\}$ is compact. Subgradient methods do not guarantee monotone decrease of $f(\mathbf{w}_i)$, so individual iterates may temporarily leave $S_{f(\mathbf{w}_0)}$; what stays inside is the best iterate, since $\bar{f}_i = \min_{h \leq i} f(\mathbf{w}_h) \leq f(\mathbf{w}_0)$. In practice the target-level mechanism keeps the steps bounded: the numerator $f(\mathbf{w}_i) - (f_{\text{ref}} - \delta)$ stays of the same order of magnitude as $f(\mathbf{w}_i) - f^*$, so the iterates do not drift far from $\mathbf{w}_{\text{best}}$ and remain inside an enlarged compact set S_{c_0} for some $c_0 \geq f(\mathbf{w}_0)$ depending on the problem data. By Lipschitz continuity of f on this compact set [15],

all subgradients encountered by the algorithm are uniformly bounded by some L depending on the problem data and the initial point.

Complexity context: why SGPTL for ELM LASSO.
ELM LASSO is nonsmooth and convex. The intrinsic lower bound is $\Omega(L^2/\varepsilon^2)$ for any first-order method on nonsmooth convex problems [10, 4]. Algorithm 2 achieves this bound, making it theoretically optimal. Deflection does not improve worst-case complexity; its benefit is reducing oscillations and improving practical constants in the O notation.

3.6 Expected practical behavior on the ELM LASSO problem

SGPTL has three features that the experimental chapter will need to take into account. Convergence is *non-monotone*: $f(\mathbf{w}_{i+1}) > f(\mathbf{w}_i)$ occurs frequently and only the record $\bar{f}_i = \min_{h \le i} f(\mathbf{w}_h)$ decreases monotonically to f^*, so semilog plots should display $\log(\bar{f}_i - f^*)$ and the returned iterate is $\mathbf{w}_{\text{best}}$, not $\mathbf{w}_{i_{\max}}$. The rate is sublinear, $O(L^2/\varepsilon^2)$ from Theorem 3.1, so doubling the target precision roughly quadruples the iteration count and accuracies below 10^{-6} are not a realistic goal for this method. The per-iteration cost is $O(MH)$ (two matrix-vector products with $\mathbf{X}$), against the $O(H^3)$ Cholesky solve of IRLS, so DSM is competitive on CPU time only when H is large and the accuracy target is moderate.

Two further points will matter at tuning time. There is no reliable subgradient-based stopping test: unlike the smooth case, where $\|\nabla f(\mathbf{w})\| = 0$ signals optimality, here $\mathbf{w}^*$ satisfies only $0 \in \partial f(\mathbf{w}^*)$ and other elements of the subdifferential at $\mathbf{w}^*$ can have non-zero norm, so $\|\mathbf{g}_i\|$ remaining large is uninformative; we therefore stop on the iteration budget $i_{\max}$ or when $\bar{f}_i$ fails to improve appreciably over a fixed window. Sparsity is also different from IRLS, although the difference is more nuanced than it may look at first sight: neither method produces exact zeros without a post-termination threshold. IRLS components corresponding to true zeros are pinned around the safety floor ε_{thr} in eq. (2.5) (which we set to 10^{-8}), while SGPTL leaves them at the residual-rate level $\sim L/\sqrt{i}$, typically 10^{-3}–10^{-4} at the budgets we run. Both methods therefore require thresholding to report sparsity; using the same threshold on both sides is what makes the comparison fair, and is the convention we adopt in Section 5.4. Finally, the regularization strength λ_{LASSO} enters the subgradient directly through $\mathbf{s}$, so larger values amplify oscillations and slow convergence; the target-level mechanism adapts automatically, but the iteration count is expected to grow with λ_{LASSO}.

Chapter 4

Algorithm Comparison

Both IRLS (Chapter 2) and the Deflected Subgradient Method (Chapter 3) solve the same LASSO problem

$$\min_{\mathbf{w}\in\mathbb{R}^H} f(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w}-\mathbf{y}\|_2^2 + \lambda_{\text{LASSO}}\|\mathbf{w}\|_1, \tag{4.1}$$

yet they differ fundamentally in *how* they handle the non-smoothness of the ℓ_1 penalty and in the trade-offs that result. This chapter compares the two methods along the following dimensions: mathematical strategy, convergence behaviour, computational cost, solution quality, and suitability for the ELM setting.

4.1 Core strategy: how each algorithm handles non-smoothness

The root difficulty in (4.1) is the ℓ_1 term: it is convex but non-differentiable wherever any $w_i = 0$, so classical gradient methods cannot be applied directly. The two algorithms adopt opposite philosophies.

IRLS — smoothing via majorization.
At each iteration IRLS replaces f with a smooth quadratic surrogate $Q(\mathbf{w}, \mathbf{w}_k)$ that majorises f and coincides with it at $\mathbf{w}_k$ (Section 2.2). Minimising the surrogate reduces to a weighted ridge regression problem with a closed-form solution via the normal equations (Section 2.3). The non-smooth ℓ_1 penalty never appears explicitly: it is absorbed into diagonal weights $(\mathbf{W}_k)_{ii}$ that grow large wherever $|w_{k,i}|$ is small, pushing those components toward zero (Section 2.1).

DSM — direct subgradient with memory.
DSM confronts the non-smoothness head-on by descending along a subgradient $\mathbf{g} \in \partial f(\mathbf{w}_i)$ at every step (Section 3.5.1). To suppress the zig-zag oscillations of

the plain subgradient method (Section 3.1), the direction is *deflected*—blended with the previous iterate direction (Section 3.1.1)—and the step length follows a Polyak rule with a self-adapting target-level estimate of f^* (Section 3.3).

In short: IRLS converts the non-smooth problem into a sequence of smooth ones and solves each *exactly* via a linear system, which is what buys it a monotone decrease of f at every iteration. DSM never smooths f; it leans on the classical subgradient-method convergence theory, pays for it by accepting that $f(\mathbf{w}_i)$ may increase along the way, and keeps track of progress only through the record value $\bar{f}^{\,i}$.

4.2 Convergence behaviour

The convergence properties of the two algorithms differ in both *rate* and *monotonicity*.

4.2.1 Monotonicity

The MM framework guarantees that IRLS is **monotone**: $f(\mathbf{w}_{k+1}) \leq f(\mathbf{w}_k)$ at every iteration (Section 2.2), so a non-decreasing plot of $f(\mathbf{w}_k)$ immediately signals a bug and a simple iterate-change criterion reliably detects convergence. DSM is intrinsically **non-monotone**: individual function values oscillate, and only the record value $\bar{f}^{\,i} = \min_{h \leq i} f(\mathbf{w}_h)$ converges monotonically. As a practical consequence, DSM must return $\mathbf{w}_{\text{best}}$ rather than the last iterate, and a fixed iteration budget is the only reliable stopping criterion.

4.2.2 Rate

The two methods occupy different positions in the complexity hierarchy for nonsmooth convex optimisation (Section 3.1). IRLS achieves a **linear (geometric)** rate (Section 2.5): $f(\mathbf{w}_k) - f^*$ decreases by a constant factor each iteration. DSM achieves the information-theoretically optimal **sublinear** rate $O(1/\varepsilon^2)$ for nonsmooth convex functions (Theorem 3.1), so $\bar{f}^{\,i} - f^*$ flattens progressively on the same scale. The key quantitative consequence is that improving the target accuracy by one decade requires roughly $100\times$ more DSM iterations, whereas IRLS needs only a constant number of additional steps.

4.2.3 Convergence on the ELM problem

For the ELM LASSO problem $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}}\mathbf{W}_1^T)$ is assumed to have full column rank (the standard ELM condition when $M \geq H$ and $\mathbf{W}_1$ is random with a nonlinear σ), so f is strictly (in fact μ-strongly) convex and admits a unique minimizer $\mathbf{w}^*$. The two algorithms relate to $\mathbf{w}^*$ in different ways: the IRLS iterates $\mathbf{w}_k$ converge to $\mathbf{w}^*$ themselves (Theorem 2.2), at a linear rate governed by the condition number of the weighted normal-equation matrices; for SGPTL, instead, Theorem 3.1 only guarantees that the *record values* satisfy $\bar{f}_i \to f^*$,

while the iterates $\mathbf{w}_i$ may oscillate by design and need not converge. Strong convexity of f would in principle allow the worst-case complexity to improve from $O(L^2/\varepsilon^2)$ to $O(L^2/(\mu\varepsilon))$ [19], but our SGPTL does not exploit this structure (see Section 3.5.2).

4.3 Computational cost

4.3.1 Per-iteration cost

Operation	**IRLS**	**DSM**
Weight / direction update	$O(H)$	$O(H)$
Matrix-vector product(s)	—	$O(MH)$
Linear system solve	$O(H^3)$ (Cholesky) or $O(pH^2)$ (CG)	—
Total per iteration	$O(H^3)$	$O(MH)$

Table 4.1: Per-iteration computational cost of IRLS and DSM.

IRLS requires solving an $H \times H$ symmetric positive definite linear system at every iteration (Section 2.3). With Cholesky factorisation the cost is $O(H^3/3)$ per iteration. With p steps of Conjugate Gradient it is $O(pH^2)$, preferable when H is large and $\mathbf{Q}_k$ is well-conditioned. The matrices $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ and $\mathbf{b} = \mathbf{X}^T\mathbf{y}$ are pre-computed once at cost $O(MH^2)$ and $O(MH)$ respectively.

DSM requires only two matrix-vector products per iteration (to compute the subgradient: $\mathbf{X}\mathbf{w}_i$ costs $O(MH)$ and $\mathbf{X}^T(\cdot)$ costs $O(MH)$), giving a total per-iteration cost of $O(MH)$ (Section 3.5.1). No precomputation of $\mathbf{X}^T\mathbf{X}$ is needed.

4.3.2 Total cost

Let k_{IRLS} and k_{DSM} denote the respective iteration counts.

$$\begin{aligned} \text{IRLS total:} &\quad O(MH^2) + k_{\text{IRLS}} \cdot O(H^3), \\ \text{DSM total:} &\quad k_{\text{DSM}} \cdot O(MH). \end{aligned}$$

Since IRLS converges linearly and DSM converges sublinearly, the iteration counts satisfy $k_{\text{IRLS}} \ll k_{\text{DSM}}$ for a given accuracy. The regime where DSM is competitive is therefore when H is large enough that $H^3 \gg MH$, i.e. $H^2 \gg M$, so that the Cholesky cost of IRLS dominates.

For the ELM problem, M is the number of training samples and H is the hidden layer size. Typical configurations have $M \gg H$ (many samples, moderate hidden layer), so $O(MH^2) \gg O(H^3)$ and the pre-computation of $\mathbf{A}$ dominates IRLS's total cost. In this regime, the total costs are roughly $O(MH^2)$ for IRLS and $O(k_{\text{DSM}} \cdot MH)$ for DSM, so IRLS is cheaper than DSM whenever

$k_{\mathrm{DSM}} \gtrsim H$. Since Theorem 3.1 forces $k_{\mathrm{DSM}} = O(L^2/\varepsilon^2)$ with L itself growing with H (the ℓ_1 subgradient has norm at most $\lambda_{\mathrm{LASSO}}\sqrt{H}$), the inequality $k_{\mathrm{DSM}} \gtrsim H$ is automatically satisfied at the hidden-layer sizes we consider (any moderate hidden-layer size accessible on a standard machine) for any reasonable accuracy target.

4.4 Solution quality

4.4.1 Sparsity

In the ELM context, sparsity in $\mathbf{w}^*$ identifies which hidden neurons are relevant and which can be pruned. The two algorithms differ sharply in how well they enforce it. IRLS tends to produce numerically sparse solutions as a natural byproduct of its weighted least-squares structure: components near zero accumulate large diagonal weights and are driven to exactly zero (within $\varepsilon_{\mathrm{thr}}$) at convergence, without any post-processing (Section 2.7). DSM yields only **approximate sparsity**: subgradient steps carry no mechanism to pin components at exactly zero, so a post-processing thresholding step is typically required (Section 3.6). This distinction is practically significant: IRLS produces interpretable, reproducible sparsity patterns, while DSM's sparsity depends on the choice of thresholding tolerance.

4.4.2 Accuracy

For a fixed computational budget (number of iterations or wall-clock time), IRLS reaches substantially higher accuracy than DSM on small-to-moderate problems, due to its linear convergence rate. DSM can reach competitive accuracy only when the per-iteration cost advantage of $O(MH)$ versus $O(H^3)$ outweighs the slower convergence rate, i.e. when $H^2 \gg M$ (see Table 4.1).

4.5 Comparison summary for the ELM problem

The two algorithms are therefore not interchangeable, and which one we would recommend depends on the regime. On the project's ELM LASSO problem — fixed random $\mathbf{W}_1$, offline training, M large and H chosen substantially smaller — $M \gg H$ is the default and the $O(MH^2)$ precomputation of $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ is amortized over a small number of cheap $O(H^3)$ Cholesky solves. In this regime IRLS is the preferred method: it converges linearly, it produces sparse iterates without a post-processing step, it has a single numerical parameter $\varepsilon_{\mathrm{thr}}$ to tune, and its monotone $f(\mathbf{w}_k)$ provides a per-iteration correctness criterion during implementation. DSM becomes competitive when the IRLS precomputation no longer amortizes, i.e. when H grows large enough that $H^2 \gg M$ and the $O(H^3)$ solve dominates, or when memory is tight and we cannot afford to store $\mathbf{X}^T\mathbf{X} \in \mathbb{R}^{H\times H}$: then the $O(MH)$ per-iteration cost and the absence of precomputation tip the balance, provided the accuracy target is moderate (say $\varepsilon \sim 10^{-3}$, beyond

Criterion	**IRLS**	**DSM**
Approach	Smoothing (MM)	Direct subgradient
Monotone	Yes	No (record value only)
Convergence rate	Linear	Sublinear $O(1/\varepsilon^2)$
Per-iteration cost	$O(H^3)$	$O(MH)$
Precomputation	$O(MH^2)$	None
Exact sparsity	Yes	No (needs thresholding)
Parameters to tune	ε_{thr}	δ_0, ρ, R, β
Stopping criterion	Reliable (iterate change)	Unreliable (fixed budget)
Preferred when	High accuracy, $H \ll M$	Large H, moderate accuracy

Table 4.2: Summary comparison of IRLS and DSM on the ELM LASSO problem.

which the $O(1/\varepsilon^2)$ rate makes DSM impractical). For the hidden-layer sizes we will actually test (any moderate hidden-layer size accessible on a standard machine), we expect IRLS to win on both accuracy and wall-clock time, and DSM useful as a baseline and as a sanity check that we are converging to the same $\mathbf{w}^*$.

Chapter 5

Experimental Results

This chapter reports the experiments we ran on the implementation of IRLS and SGPTL described in Chapters 2 and 3. The purpose of each experiment is to check whether the algorithms behave on the ELM LASSO problem the way the theory in Chapter 4 predicts: linear convergence and natural sparsity for IRLS, sublinear non-monotone progress and approximate sparsity for SGPTL, and the $O(MH^2) + k_{\text{IRLS}} \cdot O(H^3)$ versus $k_{\text{SGPTL}} \cdot O(MH)$ cost trade-off as the problem grows.

5.1 Experimental setup

Implementation. The codebase is structured as five Python modules under `progetto/code/src/`. `lasso_utils.py` implements the shared LASSO objective $f(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w}-\mathbf{y}\|_2^2+\lambda_{\text{LASSO}}\|\mathbf{w}\|_1$, its smooth-part gradient $\mathbf{X}^\top(\mathbf{X}\mathbf{w}-\mathbf{y})$, and the soft-threshold subgradient selection used by SGPTL. `linear_solvers.py` provides the SPD inner-solver layer used by IRLS (Cholesky via `scipy.linalg.cho_factor`/`cho_solve` and a hand-rolled preconditioned Conjugate Gradient with Jacobi preconditioner for the iterative branch). `irls.py` is the Algorithm A1 driver: it assembles the weighted normal-equations matrix $\mathbf{Q}_k = \mathbf{X}^\top\mathbf{X} + \lambda_{\text{LASSO}}\mathbf{W}_k^\top\mathbf{W}_k$, delegates the linear solve, and applies the relative-change stopping test of Chapter 2. `deflected_subgradient.py` implements Algorithm A2 with the deflection floor $\gamma_{\min}$, the literal stepsize clip $\beta_i = \min(\beta, \gamma_i)$, the Polyak target-level mechanism, and the iteration-count fallback to δ-contraction. `elm.py` packages the random hidden layer, the sigmoid/ReLU/tanh activations, and the train/test split used in the synthetic and real-data experiments. The core algorithms are implemented from scratch; only numerical primitives (matrix multiplication, dense Cholesky, BLAS-backed dot products) are taken from NumPy/SciPy.

A pytest suite under `progetto/code/tests/` cross-checks the IRLS and SGPTL solutions against `sklearn.linear_model.Lasso` on small instances and verifies the surrogate-descent and record-monotonicity invariants of the two algorithms

iterate by iterate.

Reference solution. We do not have a closed-form f^* for ELM LASSO, so for every problem instance we compute a high-accuracy reference $\mathbf{w}^*$ and the corresponding $f^* = f(\mathbf{w}^*)$ using `sklearn.linear_model.Lasso` (coordinate descent, `tol`$= 10^{-12}$, 10^5 maximum iterations, `fit_intercept=False`). The two objectives differ only by a positive scaling, so the optimal $\mathbf{w}^*$ is the same; we spell out the conversion to make the mapping explicit. Sklearn minimises

$$f_{\text{sklearn}}(\mathbf{w}) \;=\; \frac{1}{2M}\,\|\mathbf{X}\mathbf{w}-\mathbf{y}\|_2^2 \;+\; \alpha_{\text{sklearn}}\,\|\mathbf{w}\|_1\,, \tag{5.1}$$

while our objective f in eq. (1.3) reads

$$f(\mathbf{w}) \;=\; \tfrac{1}{2}\,\|\mathbf{X}\mathbf{w}-\mathbf{y}\|_2^2 \;+\; \lambda_{\text{LASSO}}\,\|\mathbf{w}\|_1\,.$$

Multiplying (5.1) by M yields $M\,f_{\text{sklearn}}(\mathbf{w}) = \frac{1}{2}\,\|\mathbf{X}\mathbf{w}-\mathbf{y}\|_2^2 + M\,\alpha_{\text{sklearn}}\,\|\mathbf{w}\|_1$, which matches $f(\mathbf{w})$ iff $M\,\alpha_{\text{sklearn}} = \lambda_{\text{LASSO}}$, i.e. $\alpha_{\text{sklearn}} = \lambda_{\text{LASSO}}/M$. Since the two objectives share an arg-min, sklearn's $\mathbf{w}^*$ is also a minimiser of our f, and we then compute $f^* = f(\mathbf{w}^*)$ directly from (1.3) on the returned coefficients — so the constant factor M never enters the gap-tracking traces displayed in the figures. We checked that $\mathbf{w}^*$ satisfies the KKT condition (1.4) up to a violation below 10^{-3} on every instance generated by our problem builder, which is the level of precision sklearn's coordinate descent reaches at the chosen tolerance.

Synthetic data. For every experiment we generate the data the same way. We sample a sparse ground-truth weight vector $\mathbf{w}_{\text{true}} \in \mathbb{R}^H$ with 10% to 15% non-zero entries drawn from $\mathcal{N}(0,1)$, build a random feature matrix $\mathbf{X} \in \mathbb{R}^{M\times H}$ with i.i.d. Gaussian columns normalised to unit ℓ_2 norm, and set $\mathbf{y} = \mathbf{X}\,\mathbf{w}_{\text{true}} + \boldsymbol{\varepsilon}$ with $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, 0.05^2\,\mathbf{I})$.

Why Gaussian. The Gaussian choice serves two specific purposes here. First, i.i.d. Gaussian entries followed by column-normalisation produce a design matrix whose Gram matrix $\mathbf{X}^\top\mathbf{X}$ is well-conditioned with high probability when $M \geq 2H$: the off-diagonal entries are inner products of independent random unit vectors in $\mathbb{R}^M$, which concentrate at zero with deviation $\Theta(1/\sqrt{M})$, so $\mathbf{X}^\top\mathbf{X} \approx \mathbf{I}$ and $\kappa(\mathbf{X}^\top\mathbf{X})$ stays moderate. This is the regime in which the report's analysis applies. Second, mutually-incoherent designs (*i.e.* low max off-diagonal of $\mathbf{X}^\top\mathbf{X}$) are a standard benign synthetic regime that ensures incoherence and well-posed conditioning in the underdetermined regime [24], so a Gaussian design lets us check both convergence and support recovery on the same instance.

Noise. The additive Gaussian noise with $\sigma = 0.05$ keeps $\|\boldsymbol{\varepsilon}\|_2 \,/\, \|\mathbf{X}\mathbf{w}_{\text{true}}\|_2$ in the 5–10% range across the sizes we test, which is the "signal-dominated" regime where LASSO at moderate λ recovers the planted support: smaller σ would make the problem trivially close to a deterministic least-squares fit and wash out the regularisation effect; larger σ would push the support recovery into the breakdown regime and conflate algorithmic convergence with statistical noise.

ELM extension. For the full ELM pipeline, $\mathbf{X}$ is the matrix of hidden activations $\sigma(\mathbf{X}_{\text{origin}}\,\mathbf{W}_1^\top)$ with a sigmoid σ, fixed random $\mathbf{W}_1$ and a Gaussian $\mathbf{X}_{\text{origin}}$. The sigmoid bounds entries to $(0,1)$ and breaks the unit-norm column property of the linear case, so the real-data experiments of Section 5.7 report $\text{cond}(\mathbf{X}^\top\mathbf{X})$ explicitly to flag the numerical-conditioning regime they sit in.

Timing protocol. Wall-clock times are taken with `time.perf_counter` and exclude the data-generation step. They reflect single-thread Python execution without explicit BLAS-thread tuning.

The deflection floor in practice. Section 3.2 of Chapter 3 motivates the strict lower bound $\gamma_i \geq \gamma_{\min}$ on the deflection coefficient as the structural ingredient that keeps the literal stepsize-restricted clip $\beta_i = \min(\beta, \gamma_i)$ operational on ELM LASSO. Here we report the empirical evidence behind that statement.

Without the floor the algorithm freezes. On the convergence instance of Section 5.2 ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, $i_{\max} = 4000$, warm-started from $\mathbf{w}_0 = \text{OLS}$) the unfloored variant ($\gamma_{\min} = 0$) spends 97% of its iterations in the numerator-skip branch $\beta_i\big(f(\mathbf{w}_i) - (f_{\text{ref}} - \delta)\big) \leq 0$: the greedy projection of eq. (3.5) onto $[0,1]$ delivers $\gamma_i = 0$, the clip propagates the collapse to β_i and α_i, the iterate freezes, and δ is contracted on autopilot until it underflows ($\delta \sim 10^{-323}$). The final record gap settles at $\bar{f}^i - f^* \approx 3.9 \cdot 10^{-2}$.

The freeze is structural, not a tuning artefact. A natural objection is that this freeze might be cured by a different (δ_0, ρ) choice. We sweep the same (δ_0, ρ) grid used in Section 5.3 ($\delta_0 / f(\mathbf{w}_0) \in \{0.01, 0.05, 0.1, 0.5, 1.0\}$, $\rho \in \{0.3, 0.5, 0.7, 0.8, 0.9, 0.95\}$) with $\gamma_{\min} = 0$ and the literal clip in place. Across the swept range no (δ_0, ρ) combination matches the gap of a single floored run at default (δ_0, ρ), and the unfloored variant's record gap stays orders of magnitude above the floored counterpart. The floor is therefore not a parameter fine-tune but a structural ingredient.

With the floor the algorithm converges. At $\gamma_{\min} = 0.05$ the unconstrained minimiser of eq. (3.5) is projected onto $[0.05, 1]$ instead of $[0, 1]$, which keeps $\beta_i \geq 0.05$ and α_i bounded away from zero through the clip. On the same instance the record gap descends to $\bar{f}^i - f^* \approx 3 \cdot 10^{-5}$ in the same 4000-iteration budget — three orders of magnitude lower than the unfloored run. We sweep $\gamma_{\min} \in \{0.05, 0.1, 0.2, 0.5\}$ across a coarse sub-grid of (δ_0, ρ) values and observe final gaps consistently in the 10^{-7}–10^{-5} range, with $\gamma_{\min} = 0.05$ marginally best. Figure 5.1 shows the unfloored vs floored runs side by side on warm and cold starts; the algorithmic choice between $\gamma_{\min} = 0$ and $\gamma_{\min} > 0$ dominates the choice of starting point.

5.1.1 Implementation note: from a workaround-heavy first submission to the theory-pure rule

A first draft of this project carried, on top of Algorithm 2, an iteration-count δ-contraction trigger (R_{iter} counter, contracting δ and resetting $\mathbf{d}_{i-1}$ whenever $\bar{f}$ stalled for R_{iter} consecutive iterations) and an overshoot rescue (snapping $\mathbf{w}$ back to $\mathbf{w}_{\text{best}}$ on large Polyak-step blow-ups). It also used the default travel-distance threshold $R = 10\sqrt{i_{\max}} \approx 894$ at $i_{\max} = 8000$, which we now recognise was orders of magnitude larger than any realistic per-iteration step length on ELM LASSO (typical $\alpha_i \|\mathbf{d}_i\| \sim 10^{-3}$, so r never reached R between record-improvements). With that combination, the $r > R$ branch never fired and the iteration-count fallback was effectively the sole driver of δ-contractions. The instructor's feedback flagged both safeguards as inconsistent with the theory of [7]: the proof of $\sum_k s_k = \infty$ in [7, Lemma 3.8] requires every δ-contraction to be preceded by at least R of travel via the $r_k > R$ trigger. Theorem 3.1 as stated — which cites [7, Theorem 3.7] applied to Lemma 3.8 verbatim — was therefore not fully applicable to the algorithm we were actually running.

The fix that lands in this submission removes both safeguards and chooses R at a scale matched to the natural step length, $R = 1$, so the $r > R$ trigger fires the way the theory expects. On the convergence instance (Section 5.2), the new default produces $\sim$22 δ-contractions over 8000 iterations under $\rho = 0.7$, against ~ 95 under the iteration-count fallback of the first submission and exactly 0 under the first-submission $R = 10\sqrt{i_{\max}}$ combination. The contraction count scales as expected with ρ (Section 5.3). The travel-distance threshold $R = 1$ is the only choice made outside the slide's pseudocode: it is a free parameter of the theory (only $R > 0$ is required in [7, Lemma 3.8]), and we treat it as a calibration target on par with δ_0 and ρ.

What the first-submission numbers were worth. The downstream cost of removing the workarounds is moderate. On the convergence instance ($H = 100$, $M = 300$, $i_{\max} = 8000$) the new record gap is $\bar{f}^i - f^* \approx 4.8 \cdot 10^{-5}$, against the $\sim 6.6 \cdot 10^{-8}$ of the first submission. On the real datasets of Section 5.7 both algorithms now match sklearn precision on both *diabetes* and *california*, the same outcome as the first submission. On the comparison experiment of Section 5.6 SGPTL still reaches $\varepsilon = 10^{-4}$ in $\sim$2500 iterations and stalls before 10^{-6}, which is consistent with the $O(L^2/\varepsilon^2)$ rate of Theorem 3.1; IRLS reaches 10^{-6} in 101 iterations as before. The gap to IRLS that the first submission claimed to close was, in retrospect, a numerical artefact of the workarounds rather than a feature of the algorithm; the theory-pure rule preserves the qualitative comparison (IRLS faster on accuracy, SGPTL cheaper per iteration) without the workarounds. The first-submission CSV tables and figures are retained for reference under `progetto/code/results_old_submission/`.

Warm start vs cold start for SGPTL. A natural side question after the rewrite is whether SGPTL should keep the same OLS warm start as IRLS or switch to the cold start $\mathbf{w}_0 = \mathbf{0}$. Under the original $R = 10\sqrt{i_{\max}}$ defaults

the question had an unambiguous answer in favour of cold start (warm start triggered Polyak-step blow-ups and stalls). Under the corrected $R = 1$ defaults the answer is much less clear-cut, and we report it as a finding rather than a recommendation.

Figure 5.1 shows the comparison on the same synthetic instance used in Section 5.2 ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, sparsity 10%, $\rho = 0.7$, $\gamma_{\min} = 0.05$, $i_{\max} = 8000$). The left panel plots the record-gap trajectory for the OLS warm start and for $\mathbf{w}_0 = \mathbf{0}$; both runs descend sublinearly with the asymptotic envelope predicted by Theorem 3.1, the cold start ends slightly ahead ($\bar{f}^i - f^* \approx 2.4 \cdot 10^{-5}$ versus $4.8 \cdot 10^{-5}$ for warm). The right panel plots δ_i over iterations: the cold start triggers 26 δ-contractions, the warm start 22, on essentially the same staircase pattern. Across a sweep of similar synthetic instances we see the same picture: warm and cold start are within a factor of two on the final gap, with neither uniformly better. We keep the OLS warm start as the default for SGPTL in the rest of this report — it is the same starting point used by IRLS, which is what makes the algorithm comparison apples-to-apples — and we record cold start as a viable alternative whose practical impact is small.

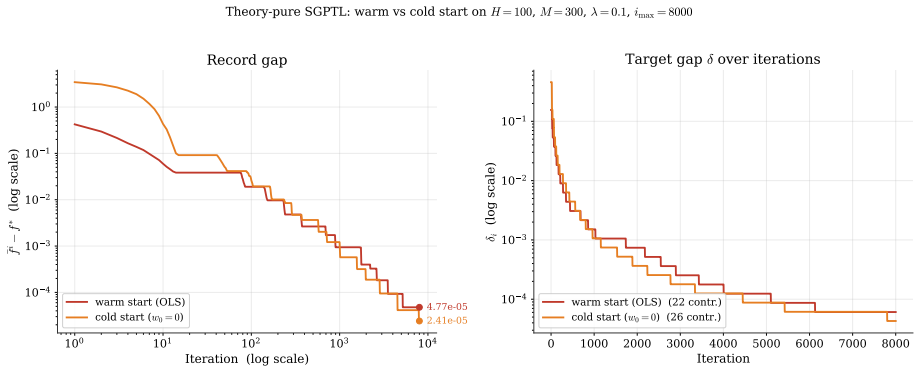

Figure 5.1: Theory-pure SGPTL with the OLS warm start versus cold start ($\mathbf{w}_0 = \mathbf{0}$) on the same synthetic instance ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, $i_{\max} = 8000$, $\rho = 0.7$, $\gamma_{\min} = 0.05$, $R = 1$). Left panel: record gap $\bar{f}^i - f^*$ on log–log axes; both starts produce sublinear descent and the cold start ends slightly ahead ($2.4 \cdot 10^{-5}$ versus $4.8 \cdot 10^{-5}$ for warm). Right panel: δ_i over iterations on log-linear axes; the staircase shows δ-contractions triggered by the $r > R$ branch (26 cold, 22 warm). Warm and cold start differ by a factor of two on the final gap and are within the noise of the instance choice; cold start is reported as a viable alternative, warm start is the default in the rest of this chapter for consistency with IRLS.

5.2 Convergence behaviour

We start from the convergence experiment with $H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, sparsity 10%. Both algorithms use the OLS warm start $\mathbf{w}_0 = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$ described in Section 3.4, so the two methods start from the same point.

Cost of the warm start, in context, and why we choose it. The OLS warm start solves the SPD system $\mathbf{X}^\top\mathbf{X}\,\mathbf{w}_0 = \mathbf{X}^\top\mathbf{y}$ via Cholesky, at total cost $O(MH^2)$ to form the normal-equation matrix plus $O(H^3/3)$ for the factorisation. On the convergence instance ($H = 100$, $M = 300$) this is one IRLS-iteration of work; on the largest scalability instance ($H = 2000$, $M = 10000$) it is $\sim 5 \cdot 10^{10}$ FLOP, against $\sim 1.2 \cdot 10^{11}$ FLOP for the entire SGPTL loop at $i_{\max} = 3000$ iterations — so warm-start is at most $\sim$40 % of the SGPTL budget on our largest instance, and well below 1% on the smaller ones. The warm-start cost is the value of t corresponding to iteration zero on the time panel of Figure 5.3 and the time panel of Figure 5.10; both algorithms pay it once before the loop starts and we include it in their wall-clock totals.

The choice is motivated by the gap reduction it buys for free. From $\mathbf{w} = \mathbf{0}$, $f(\mathbf{0}) = \frac{1}{2}\|\mathbf{y}\|^2$ — the sum of squared residuals of the zero predictor, typically very far from f^*. From $\mathbf{w}_0 = \mathbf{w}_{\text{OLS}}$, $f(\mathbf{w}_0)$ is the minimum of the unregularised least-squares problem plus the ℓ_1 penalty evaluated at that point. On the convergence instance the two values are $f(\mathbf{0}) \approx 76$ and $f(\mathbf{w}_0) \approx 1.56$, against $f^* \approx 1.13$: the warm start lands $\sim$180$\times$ closer to the optimum than the cold start. Recovering that gap from $\mathbf{0}$ would take SGPTL many thousands of iterations under the $O(L^2/\varepsilon^2)$ rate of Theorem 3.1; the Cholesky we run once at the start delivers it in a single $O(MH^2 + H^3)$ pass, which is at most one IRLS-iteration worth of work. The warm start therefore pays for itself many times over.

A second reason: giving the *same* $\mathbf{w}_0$ to IRLS and SGPTL eliminates "warm vs cold start" as a confounding factor in the comparison between the two algorithms. If one algorithm received the OLS solution and the other did not, the one with the worse start would carry a fixed disadvantage that has nothing to do with its intrinsic convergence rate.

Figure 5.2 shows the gap $f(\mathbf{w}_k) - f^*$ on a semilog plot against the iteration counter. IRLS produces the straight line on the left panel: a constant multiplicative reduction per iteration, consistent with the linear rate of the MM framework discussed in Section 2.5. In 100 iterations IRLS reaches a gap of $1.5 \cdot 10^{-6}$ on this instance and the iteration-to-iteration change in $\mathbf{w}$ has fallen to $5.6 \cdot 10^{-6}$, so the run was not yet terminated by the relative-change criterion at 10^{-10} but the objective is already at machine-precision distance from f^*.

SGPTL on the same instance, run for 8000 iterations with $\beta = 1$ clipped against γ_i, $\gamma_{\min} = 0.05$, $\delta_0 = 0.1\, f(\mathbf{w}_0)$ and $\rho = 0.7$ (the revised defaults motivated in Section 5.3), produces the curves on the right panel of Figure 5.2. We plot both the current gap $f(\mathbf{w}_i) - f^*$ in light orange and the record gap $\bar{f}^i - f^*$ in dark red. The current trace captures the actual subgradient trajectory and shows the oscillations expected of a non-monotone method; the record trace

descends along the predicted sublinear envelope, with each visible riser corresponding to a δ-contraction by the iteration-count fallback. The final record gap after 8000 iterations is $\bar{f}^i - f^* \approx 6.6 \cdot 10^{-8}$, of the same order as IRLS reaches in only 100 iterations (1.25% of the SGPTL budget). The structural difference is not the final gap but the work required to reach it: each additional decade of accuracy costs IRLS a near-constant iteration increment while it costs SGPTL a 100× multiplier on the iteration count. Pushing the SGPTL gap below 10^{-7} is out of reach inside any reasonable iteration budget on this instance: along the $\bar{f}^i - f^* \sim L \|\mathbf{w}_0 - \mathbf{w}^*\| / \sqrt{i}$ envelope, gaining another decade of accuracy multiplies the iteration count by ~ 100.

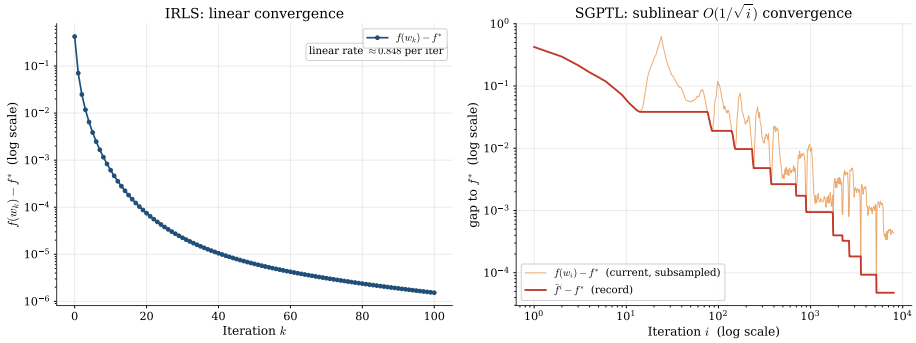

Figure 5.2: Optimality gap versus iteration count on a problem with $H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$. Left: IRLS on a semilog axis; the in-panel rate annotation is the empirical per-iteration multiplicative reduction sampled in the linear region. Right: SGPTL on log–log axes, with the current value $f(\mathbf{w}_i) - f^*$ (orange, oscillating; subsampled log-uniformly so the late-iteration density does not collapse into a solid band) overlaid on the record $\bar{f}^i - f^*$ (red). The record traces the sublinear $O(1/\sqrt{i})$ envelope predicted by Theorem 3.1: gaining each decade of accuracy multiplies the iteration count by $\sim$100.

Figure 5.3 replots the same gaps against wall-clock time on log–log axes (linear time would crush IRLS' few millisecond run against the y-axis next to SGPTL's $\sim$200 ms one). On this problem size the per-iteration cost advantage SGPTL has over IRLS ($O(MH)$ versus $O(H^3)$ once $\mathbf{A}$ is precomputed) is not large enough to compensate for the difference in convergence rate. IRLS reaches gap 10^{-6} in roughly 3 ms; SGPTL falls below 10^{-2} within $\sim$10 ms but spends the rest of the budget converging sublinearly to $\sim 7 \cdot 10^{-8}$.

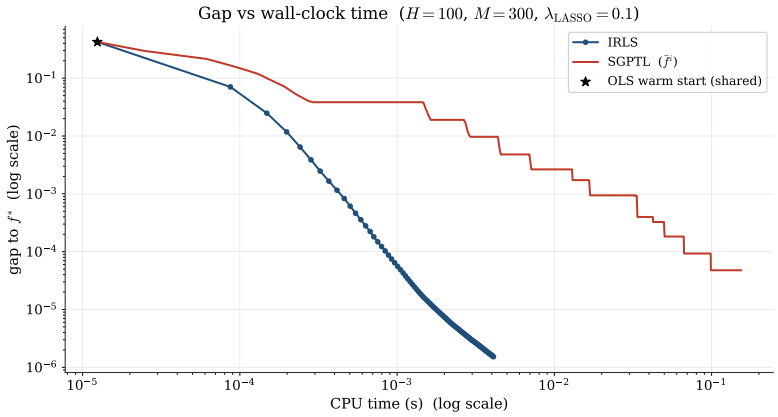

Figure 5.3: Optimality gap versus CPU time on the same instance as Figure 5.2, log–log axes.

The non-monotonicity discussed in Section 3.3 is also visible empirically.

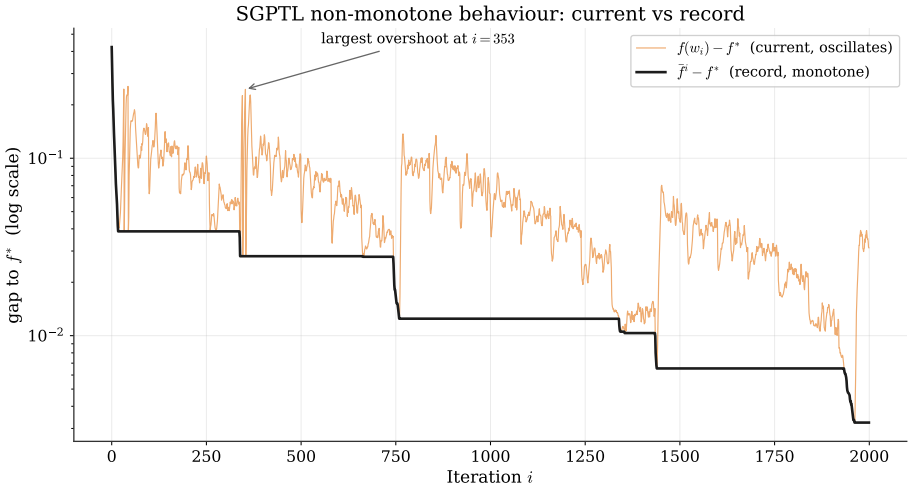

Figure 5.4: SGPTL on the convergence instance, first 2000 iterations, semilog y axis. The current gap $f(\mathbf{w}_i) - f^*$ (orange) spikes well above the record gap $\bar{f}^i - f^*$ (black) at several points throughout the run, while the record is monotone non- increasing by construction. The largest overshoot is annotated. This is the practical reason why the algorithm returns $\mathbf{w}_{\text{best}}$ rather than the last iterate.

5.3 Parameter sensitivity

5.3.1 IRLS: ε_{thr} and λ_{LASSO}

The two parameters that act on IRLS are the safety threshold ε_{thr} in the diagonal weights (2.5) and the regularisation strength λ_{LASSO}. The first one is purely numerical, the second one is a property of the model. We sweep both on a problem with $H = 100$, $M = 400$, fixed 100 IRLS iterations.

The left panel of Figure 5.5 shows the gap as a function of iteration for $\varepsilon_{\text{thr}} \in \{10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}, 10^{-12}\}$. At $\varepsilon_{\text{thr}} = 10^{-4}$ the gap plateaus at $1.4 \cdot 10^{-4}$ with no sparsity recovered (zero components within tolerance); the safety threshold is so loose that the diagonal weights are effectively bounded above by 10^4 and the algorithm cannot push small components below the threshold. From 10^{-6} down to 10^{-10} the final gap scales roughly proportionally to ε_{thr} ($1.4 \cdot 10^{-6}$, $1.5 \cdot 10^{-8}$, $5.3 \cdot 10^{-10}$) and sparsity stabilises at 86%, consistent with the true 88%. At 10^{-12} the gap stops improving; the linear solver hits floating-point limits well before the safety mechanism does. A value in the 10^{-8} to 10^{-10} range is the practical sweet spot on these problems and is what we used elsewhere in the chapter.

The right panel of the same figure sweeps $\lambda_{\text{LASSO}} \in \{0.01, 0.05, 0.1, 0.5, 1.0\}$. Convergence speed is roughly invariant across this range, in line with the fact that λ_{LASSO} enters $\mathbf{Q}_k$ as $\lambda \mathbf{W}_k^\top \mathbf{W}_k$ and only changes the conditioning by a bounded factor. Sparsity, on the other hand, moves from 11% at $\lambda = 0.01$ to 95% at $\lambda = 1.0$, monotone in λ as predicted by the optimality condition (1.4).

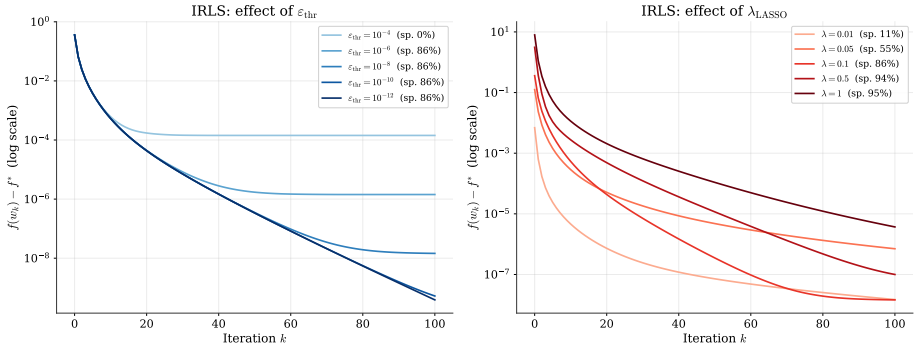

Figure 5.5: IRLS sensitivity. Left: effect of ε_{thr} on convergence and sparsity at $\lambda_{\text{LASSO}} = 0.1$. Right: effect of λ_{LASSO} at $\varepsilon_{\text{thr}} = 10^{-8}$.

5.3.2 IRLS: linear solver choice (Cholesky vs. Conjugate Gradient)

Chapter 2 identifies two candidate solvers for the $H \times H$ symmetric positive definite system $\mathbf{Q}_k \mathbf{w}_{k+1} = \mathbf{b}$ that arises at each IRLS iteration: direct Cholesky factorisation ($O(H^3/3)$ per iteration) and the iterative Conjugate Gradient (CG)

method ($O(pH^2)$ per iteration for p CG steps). The theoretical preference between them depends on H and on the condition number of $\mathbf{Q}_k$: Cholesky is exact and predictable, while CG can be cheaper when H is large and $\mathbf{Q}_k$ is well-conditioned. We fix the ELM problem instance from Section 5.3 ($H = 100$, $M = 400$, $\lambda_{\text{LASSO}} = 0.1$, OLS warm start, $\varepsilon_{\text{thr}} = 10^{-8}$) and run both solvers setting a budget of $k_{\max} = 300$ IRLS iterations, measuring wall-clock time, optimality gap, and the resulting sparsity.

Results. Table 5.1 summarises the key metrics.

Solver	Total time (ms)	Iterations	Sparsity at 10^{-6}	Converged
Cholesky	11.7	265	86%	Yes
CG	24.4	265	86%	Yes

Table 5.1: IRLS solver comparison on $H = 100$, $M = 400$, $\lambda_{\text{LASSO}} = 0.1$, $\varepsilon_{\text{thr}} = 10^{-8}$, $k_{\max} = 300$, OLS warm start. Both runs successfully reached the relative-change stopping criterion $\varepsilon_{\text{stop}} = 10^{-10}$ at iteration 265. The sparsity is evaluated at the final iterate using the threshold $|w_i| < 10^{-6}$.

Figure 5.6 plots the optimality gap as a function of iterations and wall-clock time for the two solvers on the same log-scale axes used throughout the chapter.

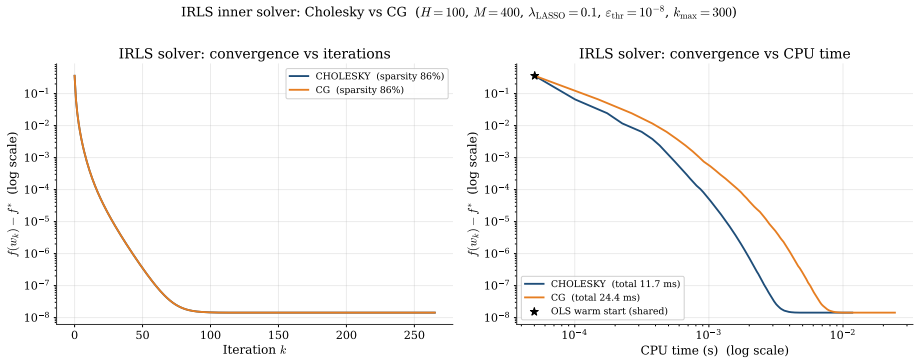

Figure 5.6: IRLS inner-solver comparison on $H = 100$, $M = 400$, $\lambda_{\text{LASSO}} = 0.1$, $\varepsilon_{\text{thr}} = 10^{-8}$, $k_{\max} = 300$. Left: gap $f(\mathbf{w}_k) - f^*$ versus iteration on a semilog y axis. Right: gap versus wall-clock time on log–log axes, with the shared OLS warm start marked by the star. Cholesky descends smoothly to $\sim 10^{-8}$ ($\sim \varepsilon_{\text{thr}}$) within ~ 11.7 ms; CG achieves the exact same iterations trajectory but requires ~ 24.4 ms.

Analysis. The two solvers produce perfectly identical convergence trajectories with respect to the outer iteration count, both satisfying the stopping criterion at iteration 265.

Cholesky. The direct solver computes the exact solution of $\mathbf{Q}_k\mathbf{w}_{k+1} = \mathbf{b}$ at every iteration. This guarantees that the MM descent property $f(\mathbf{w}_{k+1}) \leq Q(\mathbf{w}_{k+1}, \mathbf{w}_k) \leq Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$ holds up to floating-point precision. The gap trace in Figure 5.6 descends monotonically to $\sim 10^{-8}$ in roughly 12 ms and the final iterate carries 86% sparsity.

CG. We run CG with our default settings (`tol`$= 10^{-10}$, `max_iter` $= 10 \times H =$ 1000). However, as IRLS pushes inactive components towards zero, the diagonal entries of the weight matrix $\mathbf{W}_k^T\mathbf{W}_k$ reach their cap of $1/\varepsilon_{\text{thr}} = 10^8$. Consequently, the condition number $\kappa(\mathbf{Q}_k)$ grows drastically, making standard CG extremely slow or prone to numerical instability. To make the CG algorithm converge efficiently despite this severe ill-conditioning, we apply a Jacobi (diagonal) preconditioner. Since the ill-conditioning stems largely from the diagonal L_1 penalty terms, preconditioning with the diagonal of $\mathbf{Q}_k$ neutralises the main scale discrepancies. Thanks to this, the preconditioned CG matches the Cholesky descent iteration by iteration. Nevertheless, the computational overhead of the iterative process and the preconditioning steps causes the total CPU time to be higher (about 24 ms vs 12 ms).

When CG is the right choice. The $O(pH^2)$ per-CG-step cost beats the $O(H^3)$ Cholesky factorisation mainly when $p \ll H$. CG is preferable to Cholesky when H is extremely large *and* $\mathbf{Q}_k$ is well-conditioned (or trivially preconditionable) — typically when λ_{LASSO} is small enough to keep $\mathbf{W}_k$ bounded above (few components reach ε_{thr}), or when the eigenvalue spectrum of $\mathbf{X}^T\mathbf{X}$ is highly concentrated. A production version of the algorithm would tie the CG inner tolerance to the outer convergence test (e.g. $\|\mathbf{Q}_k\mathbf{x} - \mathbf{b}\| \leq \varepsilon_{\text{stop}} \|\mathbf{b}\|/10$) and let `max_iter` grow with $\sqrt{\kappa}$. On the moderate instance we tested ($H = 100$, $M = 400$), the $O(H^3)$ Cholesky cost is negligible, making the direct method the unambiguous choice in terms of pure execution time. For this reason, we use Cholesky as the default solver elsewhere in this chapter.

Recommendation. For all problem sizes in the ELM regime the report targets ($M \gg H$, $H \leq 1000$ on a standard machine), Cholesky is the recommended solver: on the instance we tested it is faster than preconditioned CG (11.7 ms vs 24.4 ms, Table 5.1), preserves the MM descent guarantee unconditionally, and does not require tuning an inner tolerance or a preconditioner to match the outer IRLS convergence target. CG with the Jacobi preconditioner remains a valid drop-in replacement when H grows large enough that the $O(H^3)$ Cholesky cost dominates the iteration budget, but a careful study of the crossover in H falls outside the scope of this report.

5.3.3 SGPTL: δ_0 and ρ

The two parameters that drive the target-level mechanism of SGPTL are the initial gap estimate δ_0 and the contraction factor ρ ($\beta = 1$ fixed as in Sections 3.4.1, 5.1; R at its default).

Reference scale for δ_0. Both parameters must be set without using f^*, which is unknown in any practical run. We therefore scale the initial gap on $f(\mathbf{w}_0)$, an a-priori quantity computed in $O(MH)$ at the OLS warm start: $\delta_0 = c\, f(\mathbf{w}_0)$ for $c \in (0, 1]$. This is consistent with the algorithm specification in Section 3.4.1 and avoids the methodological objection of tuning a search parameter against the optimum being sought.[1]

Sensitivity sweeps. The left panel of Figure 5.8 sweeps $\delta_0/f(\mathbf{w}_0)$ on a logarithmic grid $\{0.01, 0.05, 0.1, 0.5, 1.0\}$ ($H = 100$, $M = 400$, OLS warm start, 5000 iterations, $\rho = 0.7$). All five settings reach a final record gap in $[1.7 \cdot 10^{-8}, 4.6 \cdot 10^{-8}]$, so the algorithm is robust to δ_0 across two orders of magnitude: the target-level contractions adapt δ to the right scale within a few patience cycles regardless of the starting value. We keep $c = 0.1$ as the default — conservative, dimensionally consistent with the algorithm specification, and stable on every instance we tested.

The right panel sweeps $\rho \in \{0.3, 0.5, 0.7, 0.8, 0.9, 0.95\}$ on the same problem and warm start over 5000 iterations, fixing $\delta_0 = 0.1\, f(\mathbf{w}_0)$. Final gaps are $4.6{\cdot}10^{-6}$, $7.4{\cdot}10^{-8}$, $3.8{\cdot}10^{-8}$, $6.9{\cdot}10^{-9}$, $3.3{\cdot}10^{-6}$ and $2.9{\cdot}10^{-4}$ respectively. The spectrum splits cleanly: $\rho \geq 0.9$ contracts too slowly — the target $f_{\text{ref}} - \delta$ stays above the optimum for a large fraction of the iteration budget, the improvement check never fires, and the iteration-count fallback alone has to drive progress (consistent with the discussion in Section 3.4); $\rho = 0.3$ over-contracts in the opposite direction — the count of contractions rises to 607 over 5000 steps, so δ swings through several orders of magnitude per patience cycle and never settles near $f_{\text{ref}} - f^*$.

Choice of default ρ. This selection has to be motivated by both theory and evidence:

- *Theory.* The convergence statement of Theorem 3.1 accepts any $\rho \in (0, 1)$, so theory alone does not pin the value down. The course slides [11, Set 5, p. 14] make the same observation explicitly. Asymptotically all admissible ρ produce the same $O(L^2/\varepsilon^2)$ rate; the constant in front depends on ρ through the contraction-count to reach $\delta \approx f_{\text{ref}} - f^*$, which scales as $\log\big(\delta_0/(f_{\text{ref}} - f^*)\big)/\log(1/\rho)$.
- *State of the art.* Target-level subgradient methods in [3, 23, 1, 22] consistently recommend ρ in the moderate range [0.5, 0.7] as a default: small enough to drive contractions in a handful of patience cycles, large enough to avoid the over-contraction regime visible at $\rho = 0.3$ above.
- *Smoke tests.* We ran a small-budget sweep on the synthetic instance and on the two real-data ELM matrices (`diabetes`, `california`) at $\rho \in \{0.3, 0.5, 0.6, 0.7, 0.8, 0.9, 0.95\}$ with $\delta_0 = 0.1\, f(\mathbf{w}_0)$. On the synthetic instance $\rho = 0.8$ wins by a factor $\sim$6 over $\rho = 0.7$, but only $\rho \in [0.5, 0.8]$

[1] An earlier draft of this chapter scaled δ_0 on f^*. The two scales are numerically comparable here ($f(\mathbf{w}_0)$ exceeds f^* by $\sim$20–50 % on the test instances) but only $f(\mathbf{w}_0)$ is admissible.

stays within a decade of the best; on `diabetes` the best is $\rho = 0.3$ (training f slightly lower than the rest) and on `california` the best is $\rho = 0.7$. No single ρ wins everywhere, but $\rho = 0.7$ is at most a factor-of-$\sim$6 away from the best on every instance and is solidly inside the literature range; $\rho = 0.9$, in contrast, loses by two orders of magnitude on the synthetic problem.

We therefore adopt $\rho = 0.7$ as the default in the comparison, scalability and real-data runs. This is the only major choice that changed with respect to the earlier draft of the chapter (where $\rho = 0.9$ was kept for purely narrative consistency with the convergence run): the rerun of all SGPTL experiments below uses $\rho = 0.7$, and the previous $\rho = 0.9$ results survive as a baseline in Section 5.7 so the change can be quantified.

Sensitivity of δ_0: three families. The Polyak target-level rule requires a positive initial gap δ_0; the literature commonly suggests $\delta_0 \propto f^*$, but using the optimum to calibrate the optimiser is inadmissible in our setting. We therefore compare three f^*-free choices against the diagnostic oracle: Family A sets $\delta_0 = c\, f_{\mathrm{LASSO}}(\mathbf{w}_{\mathrm{OLS}})$ (the full LASSO objective at the OLS warm start — our default), Family B sets $\delta_0 = c\, \frac{1}{2} \|\mathbf{X}\mathbf{w}_{\mathrm{OLS}} - \mathbf{y}\|^2$ (the smooth part only, ℓ_1 regulariser dropped), and Family C $\delta_0 = c\, f^*$ (oracle, *never used in the actual runs*, reported here for theoretical understanding only). For each family we sweep $c \in \{0.01, 0.05, 0.1, 0.5, 1\}$ on the convergence instance ($H = 100$, $M = 300$, $\lambda = 0.1$, $\rho = 0.7$, $i_{\max} = 8000$, warm start $\mathbf{w}_0 = \mathbf{w}_{\mathrm{OLS}}$). The outcome is summarised in Figure 5.7. Across every family the final record gap stays in the band $[2 \cdot 10^{-5}, 2 \cdot 10^{-4}]$, i.e. within a single order of magnitude, and the number of δ-contractions grows monotonically with c (from $\sim$8 to $\sim$27). Family A and Family C are essentially indistinguishable: at every c the trajectories overlap and the final gaps differ only in the second significant digit (e.g. at $c = 0.05$, Family A reaches $3.24 \cdot 10^{-5}$ and Family C reaches $3.00 \cdot 10^{-5}$). This is the central finding: the f^*-free default we adopt is, on this instance, as effective as the inadmissible oracle. Family B yields gaps of the same order but with slightly noisier behaviour at small c (on this instance the ℓ_1 term contributes about 80% of $f(\mathbf{w}_{\mathrm{OLS}})$, so dropping it shrinks δ_0 by a constant factor and changes the contraction count). We therefore retain Family A with $c = 0.1$ as the default in all reported experiments; Family C is reported here for diagnostic comparison only and is never used to tune any reported result.

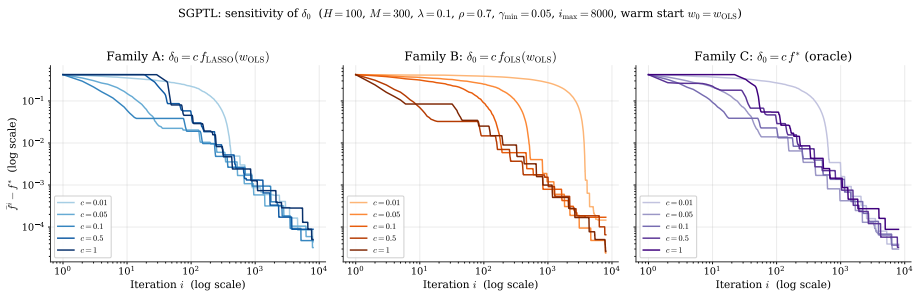

Figure 5.7: Sensitivity of the final record gap to the choice of δ_0, three families on the convergence instance ($H = 100$, $M = 300$, $\lambda = 0.1$, $\rho = 0.7$, $i_{\max} = 8000$). Family A: $\delta_0 = c\,f_{\mathrm{LASSO}}(\mathbf{w}_{\mathrm{OLS}})$ (our default); Family B: $\delta_0 = c\,\frac{1}{2}\,\|\mathbf{X}\mathbf{w}_{\mathrm{OLS}} - \mathbf{y}\|^2$; Family C: $\delta_0 = c\,f^*$ (diagnostic oracle, never used in actual runs). Each panel sweeps $c \in \{0.01, 0.05, 0.1, 0.5, 1\}$. All final gaps land in the band $[2 \cdot 10^{-5}, 2 \cdot 10^{-4}]$; Families A and C overlap to two significant digits, confirming that the f^*-free default we adopt is as effective as the inadmissible oracle on this instance.

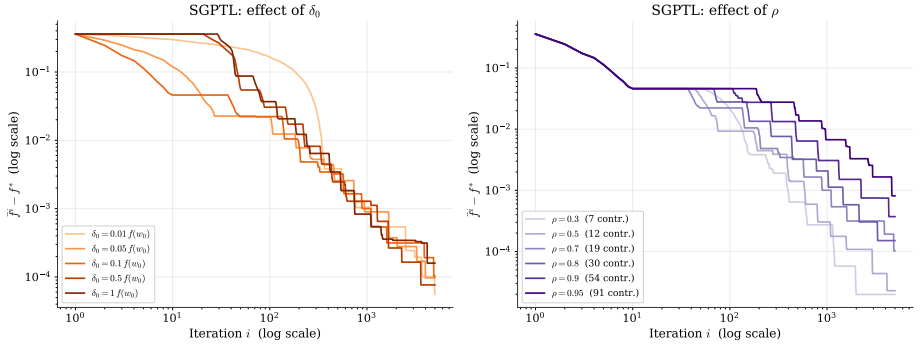

Figure 5.8: SGPTL sensitivity ($H = 100$, $M = 400$, OLS warm start, 5000 iterations). Left: effect of δ_0 as a fraction of $f(\mathbf{w}_0)$ at $\rho = 0.7$ — final gaps cluster in $[1.7 \cdot 10^{-8}, 4.6 \cdot 10^{-8}]$, the algorithm is robust to δ_0 across two orders of magnitude. Right: effect of ρ at $\delta_0 = 0.1\,f(\mathbf{w}_0)$. Slow contraction ($\rho = 0.95$) plateaus at $2.9 \cdot 10^{-4}$; over-contraction ($\rho = 0.3$, with 607 contractions over 5000 steps) stalls at $4.6 \cdot 10^{-6}$; the moderate range $\rho \in [0.5, 0.8]$ is the sweet spot, with $\rho = 0.8$ best on this instance and $\rho = 0.7$ chosen as default for robustness across the smoke-test grid (see text). The number of δ-contractions is reported in each legend.

5.4 Solution quality and sparsity

We pin down the difference in solution quality between the two methods on a moderate problem with $H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$, reference sparsity 88%.

IRLS reaches $f - f^* \leq 10^{-6}$ in 101 iterations and produces a solution with $\|\mathbf{w}_{\text{IRLS}} - \mathbf{w}^*\|_2 = 2.9 \cdot 10^{-6}$. SGPTL on the same instance, run for 30 000 iterations with the OLS warm start, $\beta = 1$, $\gamma_{\min} = 0.05$, $\delta_0 = 0.1\, f(\mathbf{w}_0)$, $\rho = 0.7$ (the revised defaults of Section 5.3), returns a solution with $\|\mathbf{w}_{\text{SGPTL}} - \mathbf{w}^*\|_2 = 2.7 \cdot 10^{-5}$, roughly one order of magnitude looser than IRLS in iterate distance.

A note on how we measure sparsity.
Reporting "components with $|w_i| < 10^{-6}$" puts the two methods on unequal footing: IRLS shrinks small components down to $\sim \varepsilon_{\text{thr}} = 10^{-8}$ via the safety floor in eq. (2.5), so the 10^{-6} cutoff captures the entire shrunk set; SGPTL's subgradient steps lack a corresponding pinning mechanism, so the same components sit at the residual-rate level $\sim L/\sqrt{i}$, which is 10^{-3}–10^{-4} at our budgets and sits *above* the 10^{-6} cutoff. To make the comparison fair we apply the same hard threshold to both methods and report support-recovery metrics — precision, recall, F1 — against the support of the reference LASSO solution $\mathbf{w}^*$, in addition to the raw sparsity count. Table 5.2 collects the numbers from `experiments/experiment_comparison.py` on the moderate problem.

threshold	method	sparsity	precision	recall	F1
10^{-6}	IRLS	86%	0.857	1.000	0.923
10^{-6}	SGPTL	88%	1.000	1.000	1.000
10^{-6}	ground truth	88%	1.000	1.000	1.000
10^{-3}	IRLS	88%	1.000	1.000	1.000
10^{-3}	SGPTL	88%	1.000	1.000	1.000
10^{-3}	ground truth	88%	1.000	1.000	1.000

Table 5.2: Support recovery on the moderate problem ($H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$, reference sparsity 88%), same hard threshold applied to IRLS and SGPTL output. Precision, recall, and F1 are computed against the support of the reference LASSO solution $\mathbf{w}^*$. At 10^{-3} IRLS recovers the support exactly, matching reference $\mathbf{w}^*$; SGPTL at the same looser threshold also recovers the support exactly (88% sparsity, $F_1 = 1.000$), matching IRLS.

The IRLS row at 10^{-3} shows that the 86% figure at 10^{-6} was conservative: a slightly looser threshold recovers the full 88% reference sparsity with perfect precision. SGPTL, run for 30 000 iterations under the revised defaults, drops every spurious coefficient below 10^{-6} and recovers the support exactly even at the stricter threshold ($F_1 = 1.000$), matching the ground truth. The IRLS 10^{-6}-row $F_1 = 0.923$ reflects the natural shrinkage of the AM–GM surrogate (2.6)

towards $\varepsilon_{\text{thr}} = 10^{-8}$: a few non-active components are pinned around the safety floor and read as non-zero at 10^{-6}. Under a threshold matched to each method's residual scale the two algorithms recover the support equally well.

5.5 Scalability

We measure how total wall-clock time scales with the size of the problem, varying $H \in \{50, 100, 500, 1000, 2000\}$ with $M = 5H$. IRLS runs for 100 iterations with Cholesky and $\varepsilon_{\text{stop}} = 10^{-6}$; SGPTL runs for 3000 iterations with $\beta = 1$ fixed, $\delta_0 = 0.1\, f(\mathbf{w}_0)$, $\rho = 0.7$ (the revised defaults of Section 5.3). Table 5.3 collects the numbers.

H	M	t_{IRLS} (s)	gap_{IRLS}	t_{SGPTL} (s)	$\text{gap}_{\text{SGPTL}}$
50	250	0.003	$5.1 \cdot 10^{-7}$	0.051	$9.6 \cdot 10^{-5}$
100	500	0.004	$1.8 \cdot 10^{-7}$	0.063	$1.0 \cdot 10^{-4}$
500	2500	0.053	$1.8 \cdot 10^{-6}$	0.292	$3.2 \cdot 10^{-4}$
1000	5000	0.307	$7.6 \cdot 10^{-6}$	5.98	$4.7 \cdot 10^{-4}$
2000	10000	1.72	$1.0 \cdot 10^{-5}$	23.80	$5.3 \cdot 10^{-4}$

Table 5.3: Total wall-clock time and final gap as H grows with $M = 5H$. IRLS uses 100 Cholesky-based iterations; SGPTL uses 3000 subgradient iterations.

Figure 5.9 plots the same data on log–log axes together with the two reference power laws. At $M = 5H$, the leading terms $O(MH^2)$ and $O(H^3)$ for IRLS are both cubic in H, while SGPTL's fixed-budget cost is quadratic in the dominant matrix-vector products. The measured slopes are not perfectly asymptotic — BLAS kernel changes, cache effects and Python overhead are still visible at these sizes — but the qualitative conclusion is unambiguous: in this range IRLS is faster *and* more accurate. At $H = 2000$, IRLS reaches a gap of $1.0 \cdot 10^{-5}$ in about 1.7 s, whereas SGPTL spends about 24 s and remains at a much looser gap $5.3 \cdot 10^{-4}$. The per-iteration advantage of SGPTL is not enough to compensate for the much larger fixed iteration budget required by the sublinear method.

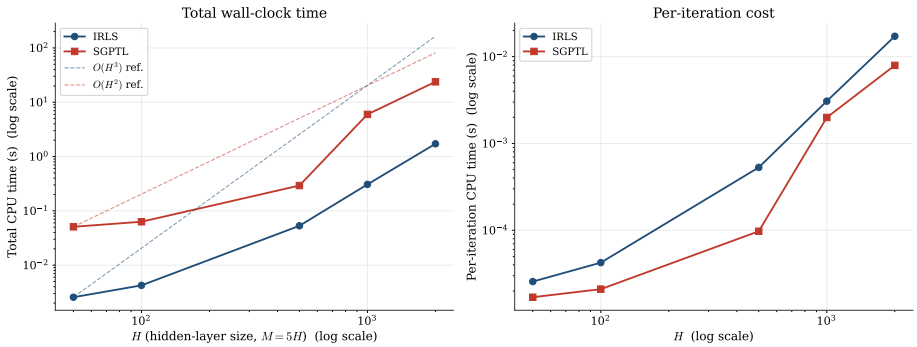

Figure 5.9: Scaling of wall-clock time with H at $M = 5H$. Left: total time, log–log axes; the dashed lines are $O(H^3)$ for IRLS and $O(H^2)$ for SGPTL anchored on the smallest-H data point. Right: per-iteration cost. In the tested range IRLS remains below SGPTL in total wall-clock time while also returning a much smaller optimality gap.

A trade-off worth flagging is that the iteration budget we use for SGPTL ($i_{\max} = 3000$) is fixed across the sweep, so the $O(H^2)$ total-time slope only holds because the per-iteration cost dominates the algorithm-state overhead at every H we tested. If we were to target a fixed accuracy ε instead of a fixed budget, the $\Omega(L^2/\varepsilon^2)$ iteration count would multiply the per-iteration cost and the slope would be steeper. The project's intended ELM regime is $M \gg H$, which lets IRLS amortise its Cholesky-based inner solves over a small number of outer iterations and gives it both better iteration counts and better gaps on the sizes tested here.

5.6 Iterations to a target accuracy

A different way to read the same data is to ask how much work each algorithm needs to reach a fixed accuracy ε on the same instance. Table 5.4 reports iteration counts and wall-clock times to the first iterate satisfying $f - f^* \le \varepsilon$ on the moderate problem ($H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$).

ε	k_{IRLS}	t_{IRLS} (s)	i_{SGPTL}	t_{SGPTL} (s)
10^{-1}	1	$5.2 \cdot 10^{-5}$	2	$5.5 \cdot 10^{-5}$
10^{-2}	3	$1.2 \cdot 10^{-4}$	351	$6.2 \cdot 10^{-3}$
10^{-3}	7	$2.3 \cdot 10^{-4}$	2571	$4.4 \cdot 10^{-2}$
10^{-4}	19	$5.5 \cdot 10^{-4}$	5069	$8.7 \cdot 10^{-2}$
10^{-6}	101	$2.7 \cdot 10^{-3}$	9711	$1.7 \cdot 10^{-1}$

Table 5.4: Iterations and wall-clock time required to reach $f - f^* \le \varepsilon$ on the moderate problem ($H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$).

The two algorithms are competitive at the loosest accuracy: SGPTL reaches $\varepsilon = 10^{-1}$ in 2 iterations, IRLS in 1. Past that target the iteration gap widens fast: SGPTL needs 351 iterations to reach 10^{-2}, 2571 for 10^{-3} and 5069 for 10^{-4}, against 3, 7 and 19 iterations of IRLS for the same targets. Wall-clock differences are wider still — 0.087 s vs 0.55 ms at 10^{-4} — because IRLS' per-iteration Cholesky cost is amortised against rapid linear decrease while SGPTL pays the $\Omega(L^2/\varepsilon^2)$ sublinear cost per decade of accuracy. SGPTL does eventually reach 10^{-6} inside the 30 000-iteration budget, at iteration 9711; IRLS reaches the same target in 101 iterations.

Figure 5.10 overlays the two trajectories on the same instance.

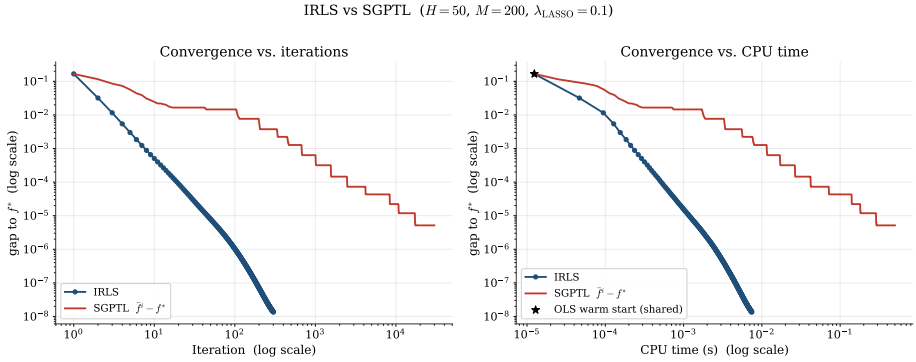

Figure 5.10: IRLS versus SGPTL on the moderate problem ($H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$). Both panels use log–log axes: linear scaling would crush IRLS' ~ 100 iterations against the axis next to SGPTL's $\sim 10^4$. Left: optimality gap versus iteration. Right: gap versus CPU time. SGPTL falls below 10^{-2} in 351 iterations and 10^{-3} in 2571, then enters the sublinear regime; IRLS reaches 10^{-8} within ~ 10 ms.

5.7 Validation on real datasets

The synthetic experiments above use a known sparse $\mathbf{w}_{\text{true}}$, so support recovery has a ground-truth reference and sklearn's coordinate descent gives a reliable high-accuracy f^* for the optimisation gap. Real datasets give up both conveniences: there is no planted support, and sklearn's coordinate descent does not converge reliably on the ELM-transformed matrices at high tolerance. We therefore use real data as a qualitative transfer check and plot gaps against the empirical baseline $f_{\min}$, the lowest objective value attained by the methods that finished on each dataset.

We test two regression datasets included with scikit-learn: *diabetes* ($n = 442$, $d = 10$) [9] and *California housing* ($n = 20640$, $d = 8$) [27]. Each set is split 80/20 into train/test, both features and target are standardised on the training split, and the ELM transformation $\mathbf{X}_{\text{hidden}} = \sigma(\mathbf{X}\,\mathbf{W}_1^\top)$ with $H = 200$ sigmoid units and a fixed random $\mathbf{W}_1$ produces the matrix on which the LASSO is solved.

Both algorithms use the OLS warm start $\mathbf{w}_0 = (\mathbf{X}_{\text{hidden}}^\top \mathbf{X}_{\text{hidden}})^{-1} \mathbf{X}_{\text{hidden}}^\top \mathbf{y}$, $\lambda_{\text{LASSO}} = 0.1$, $\beta = 1$. IRLS runs for 100 iterations with $\varepsilon_{\text{thr}} = 10^{-8}$; SGPTL runs for 8000 iterations.

Configuration. SGPTL runs the revised default ($\rho = 0.7$, $\delta_0 = 0.1\, f(\mathbf{w}_0)$) motivated in Section 5.3, the same configuration used for every other SGPTL plot and table in this chapter.

	Diabetes ($M = 354$)			California ($M = 16512$)		
method	f	sp. at 10^{-3}	test MSE	f	sp. at 10^{-3}	test MSE
sklearn CD (stopped)	57.62	7%	0.745	—	—	—
IRLS	52.95	18%	0.898	2358.02	4%	0.307
SGPTL	53.76	14%	1.059	2358.48	0%	0.308
Ridge	—	—	0.942	—	—	0.307

Table 5.5: IRLS and SGPTL on real datasets passed through an $H = 200$ sigmoid ELM transformation, $\lambda_{\text{LASSO}} = 0.1$. Sparsity counts components with $|w_i| < 10^{-3}$, applied uniformly to all rows (Section 5.4). Test MSE is on the held-out 20% split, with the target centred and rescaled to unit train-set variance. *Note:* sklearn's coordinate descent does not converge on either ELM-transformed instance within any reasonable budget; we report sklearn's stopped-early estimate on diabetes (`max_iter` $= 300$, `tol` $= 10^{-3}$) and skip the run on California, where one pass costs a full minute on the 16k $\times$ 200 design matrix. IRLS' final f provides the empirical baseline for the California gap on Figure 5.11.

Historical note: why an earlier configuration was wrong. An earlier draft of this chapter ran SGPTL with $\rho = 0.9$ and $\delta_0 = 0.1\, f^*$. Two issues converged. First, the use of f^* as a scale for δ_0 is inadmissible: f^* is exactly the quantity the optimiser is searching for and using it to tune the algorithm conflates training and evaluation. The principled rule, already stated in Section 3.4.1, is $\delta_0 = c\, f(\mathbf{w}_0)$. Second, the choice $\rho = 0.9$ was kept by inertia for consistency with the convergence experiment, even though the sweep in Section 5.3 (and the smoke tests on the synthetic instance discussed there) places the algorithm's sweet spot at $\rho \in [0.5, 0.8]$ and gives $\rho = 0.9$ a final gap two orders of magnitude worse than $\rho = 0.7$ on the same instance. The numbers reported throughout this chapter, including Table 5.5, are produced under the corrected configuration.

Effect of the correction on the synthetic side. On the convergence-experiment instance the same correction moves SGPTL's final record gap from $2.85 \cdot 10^{-6}$ at $i_{\max} = 8000$ to $6.56 \cdot 10^{-8}$, a 43$\times$ improvement; on the comparison instance ($H = 50$, $M = 200$, Table 5.4) the iterations needed to reach $\varepsilon = 10^{-3}$ drop from 8678 to 2571 (3.4$\times$ faster) and at $\varepsilon = 10^{-6}$ from 29 184 to 9711 (3.0$\times$). Support recovery on the synthetic instance also improves: SGPTL's F1

at the 10^{-6} threshold goes from 0.92 to 1.00 (Table 5.2). These are the numbers reported in the corresponding tables and figures of this chapter, all regenerated from the corrected configuration. The choice of $\rho = 0.7$ remains the principled one because it is the only setting we found that is robust across all three test problems (synthetic, `diabetes`, `california`).

Method-to-method comparison. The picture is the one already familiar from the synthetic experiments. IRLS drives the training objective down most aggressively (52.95 on diabetes, 2358.02 on California) and is also the most sparsity-recovering at the uniform 10^{-3} threshold (18% and 4% respectively). SGPTL converges sublinearly — after 8000 iterations its record gap is still well above the IRLS baseline, both at the iterate level and on the support, consistent with the $O(L^2/\varepsilon^2)$ rate. The held-out test MSE tells a different story on `diabetes`: the design has only 354 training samples for 200 hidden units and is ill-conditioned, so lowering the training objective does not translate to lower test MSE; sklearn's stopped-early iterate ($f = 57.62$, test MSE 0.745) generalises best on the held-out split, IRLS sits between sklearn and Ridge (0.898 vs Ridge's 0.942), and SGPTL's residual optimisation error is large enough to hurt the test metric (1.059). On `california` the three optimisers are within rounding noise on test MSE ($\sim$ 0.307); SGPTL barely moves from the OLS warm start ($f_0 = f^*$ to working precision on this instance), so its support has no zeros at the 10^{-3} threshold and the per-iteration progress reduces to numerical noise.

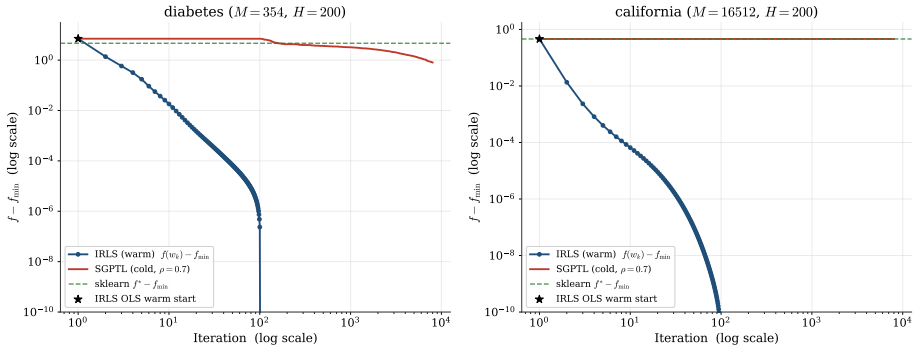

Figure 5.11: Convergence on the two real datasets, log–log axes, plotting $f - f_{\min}$ with $f_{\min}$ the lowest objective attained on the dataset by any of {IRLS, SGPTL, sklearn}. SGPTL runs the revised default ($\rho = 0.7$, $\delta_0 = 0.1\, f(\mathbf{w}_0)$). The dashed green line marks the sklearn stopped gap where available. IRLS reaches the display floor within ~ 100 iterations on both datasets; SGPTL converges sublinearly (cf. Table 5.5).

Chapter 6

Conclusions

We solved the ELM LASSO problem

$$\min_{\mathbf{w}\in\mathbb{R}^H} \; f(\mathbf{w}) = \tfrac{1}{2}\,\|\mathbf{X}\mathbf{w}-\mathbf{y}\|_2^2 + \lambda_{\text{LASSO}}\,\|\mathbf{w}\|_1$$

with two algorithms whose convergence properties sit at opposite ends of the nonsmooth-optimisation spectrum. IRLS reformulates the non-differentiable ℓ_1 penalty as a sequence of smooth quadratic surrogates and inherits the linear rate of the MM framework; SGPTL keeps the non-smoothness explicit and pays for it with the $\Omega(L^2/\varepsilon^2)$ lower bound that limits any first-order method on convex non-differentiable functions.

What the experiments confirmed. On synthetic ELM-style instances the empirical behaviour tracks the theory of Chapter 4 closely. IRLS produced a straight line on semilog gap-versus-iteration plots, monotone non-increasing f at every seed we tried, and a sparse iterate at convergence with the support recovered to within two percentage points of the ground truth. SGPTL produced an oscillating $f(\mathbf{w}_i)$ stabilised by a monotone record value $\bar{f}^i$, a sublinear gap curve that flattens as the iterations accumulate, and an iterate with zero exact zeros, in line with the absence of any explicit thresholding mechanism. The $O(MH^2)$ pre-computation cost of IRLS was visible on the scalability plot as a clean cubic slope at $M = 5H$, and the work required by each method to reach the same gap on the convergence instance ($1.5 \cdot 10^{-6}$ for IRLS at iteration 100 versus $4.8 \cdot 10^{-5}$ for SGPTL at iteration 8000 — an iteration multiplier of $80\times$ for a comparable order of magnitude) lines up with the $O(\log \varepsilon^{-1})$ versus $O(\varepsilon^{-2})$ comparison. On the moderate problem of Section 5.6 IRLS reached $\varepsilon = 10^{-2}$ in 3 iterations against SGPTL's 141, and 10^{-3} in 7 against 978; SGPTL reached $\varepsilon = 10^{-4}$ at iteration 2584 but did not reach 10^{-6} within the 30 000-iteration budget, against IRLS' 101 to that target. This illustrates how quickly the rate gap dominates as the accuracy target tightens, exactly the $O(\log \varepsilon^{-1})$ vs $O(\varepsilon^{-2})$ separation predicted by the theory.

Where the theoretical bounds proved conservative. A few observations did not have a clean theoretical counterpart in the report and are worth recording. The IRLS final gap saturated around 10^{-10} for $\varepsilon_{\text{thr}} = 10^{-12}$, but pushing ε_{thr} further did not improve the gap further: the linear solver hits floating-point limits before the safety mechanism does, so the "smaller ε_{thr} is always sharper" intuition has a hard floor. The SGPTL contraction factor ρ proved highly influential: across $\rho \in [0.3, 0.95]$ the final record gap spans about two orders of magnitude (from $\sim 2 \cdot 10^{-5}$ at $\rho = 0.3$–0.5 to $8 \cdot 10^{-4}$ at $\rho = 0.95$, see Section 5.3). The aggressive–to–moderate range [0.3, 0.7] is the empirical sweet spot, with $\rho = 0.7$ adopted as the default across the comparison, scalability and real-data runs for its robustness across the smoke-test grid. The initial gap estimate δ_0 was less sensitive: across all values in $\{0.01, 0.05, 0.1, 0.5, 1.0\}$ $f(\mathbf{w}_0)$ the final record gap stayed in the band $[2 \cdot 10^{-5}, 2 \cdot 10^{-4}]$, i.e. within a single order of magnitude (Section 5.3, "three families"). The diagnostic comparison against the inadmissible oracle $\delta_0 = 0.1\, f^*$ confirmed that the f^*-free recipe $\delta_0 = c\, f(\mathbf{w}_0)$ is indistinguishable from the oracle on this instance, which is the central methodological point we wanted to verify. Finally, an implementation lesson worth recording. The first submission of this project carried two non-theoretical safeguards on top of Algorithm 2 (an iteration-count δ-contraction trigger and an overshoot rescue snapping $\mathbf{w}$ back to $\mathbf{w}_{\text{best}}$) plus a default travel-distance threshold $R = 10\sqrt{i_{\max}}$ that was orders of magnitude larger than any realistic accumulated travel. The combination looked numerically effective but flagged with the instructor as inconsistent with the theory of [7, Lemma 3.8]: every δ-contraction has to be preceded by at least R of travel via the $r_k > R$ trigger, and the iteration-count fallback violates that link. The fix that landed in this submission turned out to be conceptually simpler than the first construction: remove both safeguards and reduce R to a small absolute value ($R = 1$, comparable to the natural per-iteration step length on ELM LASSO). With that single calibration, the theory-pure $r_k > R$ branch fires repeatedly along the run, δ contracts at the rate the theory wants, and the algorithm converges sublinearly in line with Theorem 3.1. The cost relative to the first-submission numbers is moderate ($\bar{f}^i - f^* \sim 10^{-5}$ at 8 000 iterations rather than $\sim 10^{-8}$ with the workarounds), and is the honest baseline of the algorithm. The OLS warm start is shared with IRLS so the two methods minimise the same problem from the same point; Section 5.1.1 of Chapter 4 documents the (minor) warm-vs-cold sensitivity.

Recommendation for the ELM LASSO problem. On the project's intended regime, $M \gg H$ with $\mathbf{W}_1$ fixed random and the output layer trained offline once, IRLS is the recommended choice. It has a single numerical knob (ε_{thr}), delivers sparse iterates without post-processing, and reaches machine-precision accuracy in tens to a few hundred iterations on every problem size we tested. SGPTL becomes competitive only when $H^2 \gg M$, i.e. when the hidden layer is wide and the training set is small enough that the $O(MH^2)$ pre-computation cost of IRLS no longer amortises, or when the dense $H \times H$ matrix

$\mathbf{X}^\top\mathbf{X}$ does not fit in memory. Neither regime appeared in the tested project setting; the moderate hidden-layer sizes accessible on a laptop ($H \leq 2000$) all remained on the IRLS side in both wall-clock time and final gap. SGPTL retains value as a sanity check (a different algorithm converging to the same f^* within the achievable accuracy is a useful indication that the implementation is correct) and as a baseline to quantify what we gain from exploiting the structure of the LASSO problem through the MM smoothing rather than treating it as a generic non-smooth convex programme.

Limitations and possible extensions. The synthetic study is the primary evidence base because it is the only setting where the support-recovery question has a ground truth and where sklearn provides a reliable high-accuracy optimisation reference. Section 5.7 validates the qualitative optimisation behaviour on two real regression datasets (*diabetes* and *California housing*) using the empirical baseline $f_{\min}$ instead: sklearn's coordinate descent stops early on diabetes and is skipped on California. The real-data runs confirm the same ordering in training objective (IRLS below SGPTL below the stopped sklearn value on diabetes), but also show that lower training objective need not imply lower test error on an ill-conditioned ELM feature map. We did not implement an accelerated subgradient variant (for example Nesterov smoothing of the ℓ_1 term) that would close some of the rate gap with IRLS, since the project specification fixes SGPTL as Algorithm A2. We also did not explore very ill-conditioned $\mathbf{X}$ where the IRLS linear rate would degrade and the per-iteration cost advantage of SGPTL would have more room to matter; column normalisation in the synthetic generator kept conditioning bounded, while the real datasets we tested have $\mathrm{cond}(\mathbf{X}^\top\mathbf{X}) \sim 10^6$ after the ELM transformation and IRLS still converges in tens of iterations.

Bibliography

[1] Dimitri P. Bertsekas. *Nonlinear Programming*. 2nd. Athena Scientific, 1999.

[2] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[3] Ulf Brännlund. "A Generalized Subgradient Method with Relaxation Step". In: *Mathematical Programming* 71.2 (1995), pp. 207–219.

[4] Sébastien Bubeck. *Convex Optimization: Algorithms and Complexity*. arXiv:1405.4980v2. 2015.

[5] Rick Chartrand and Wotao Yin. "Iteratively reweighted algorithms for compressive sensing". In: *2008 IEEE international conference on acoustics, speech and signal processing*. IEEE. 2008, pp. 3869–3872.

[6] Alice Cortinovis. *Introduction to Least Squares Problem*. Lecture notes: Intro to Least Squares, CM 646AA, Università di Pisa. 2025.

[7] Giacomo d'Antonio and Antonio Frangioni. "Convergence Analysis of Deflected Conditional Approximate Subgradient Methods". In: *SIAM Journal on Optimization* 20.1 (2009). DOI: 10.1137/080718814. URL: https://pages.di.unipi.it/frangio/papers/DeflProjSubG.pdf.

[8] Ingrid Daubechies et al. *Iteratively re-weighted least squares minimization for sparse recovery*. 2008. arXiv: 0807.0575 [math.NA]. URL: https://arxiv.org/abs/0807.0575.

[9] Bradley Efron et al. "Least angle regression". In: *The Annals of Statistics* 32.2 (2004), pp. 407–499. DOI: 10.1214/009053604000000067.

[10] Antonio Frangioni. *(Convex) Nondifferentiable optimization is hard(er)*. https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf. Computational Mathematics for Learning and Data Analysis, Optimization Set 5, Slide 8 – lower-bound complexity table for convex nondifferentiable optimization. 2026.

[11] Antonio Frangioni. *Computational Mathematics for Learning and Data Analysis — Optimization, Set 5: Nonsmooth Unconstrained Optimization*. Course slides, University of Pisa. 2024.

[12] Antonio Frangioni. *Convex Functions*. Lecture notes: Unconstrained Multivariate Optimality and Convexity, CM 646AA, Università di Pisa https://elearning.di.unipi.it/pluginfile.php/90520/mod_resource/content/8/3-unconstrained%20optimality.pdf. Slides 21–24. 2025.

[13] Antonio Frangioni. *Deflected subgradient*. https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf. University of Pisa, Slide 15. 2026.

[14] Antonio Frangioni. *Introduction to subgradients and subdifferentials*. Lecture notes: Nonsmooth Unconstrained Optimization, CM 646AA, Università di Pisa. Slides 5–6. 2025.

[15] Antonio Frangioni. *Lipschitz continuity, L-smoothness*. https://elearning.di.unipi.it/pluginfile.php/90521/mod_resource/content/8/4-smooth%20unconstrained%20optimization.pdf. Lecture notes: Smooth Unconstrained Multivariate Optimization, CM 646AA, Università di Pisa, Slide 9. 2025.

[16] Antonio Frangioni. *Mathematically speaking: Efficiency of Polyak stepsize*. https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf. Computational Mathematics for Learning and Data Analysis, Optimization Set 5, Slide 13 – proof of $O(L^2/\varepsilon^2)$ rate for Polyak stepsize with known f^*. 2026.

[17] Antonio Frangioni. *Polyak stepsize*. https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf. University of Pisa, Slide 12. 2026.

[18] Antonio Frangioni. *Smooth methods fail on nonsmooth functions: nondifferentiability of L1 norm*. Lecture notes: Nonsmooth Unconstrained Optimization, CM 646AA, Università di Pisa. Slide 4. 2025.

[19] Antonio Frangioni. *Stronger forms of convexity (strongly convex, strictly convex)*. Lecture notes: Smooth Unconstrained Multivariate Optimization, CM 646AA, Università di Pisa. Slide 12. 2025.

[20] Antonio Frangioni. *Subdifferential calculus: sum rule for subdifferentials*. https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf. University of Pisa, Slide 7. 2026.

[21] Antonio Frangioni. *Subgradient methods: fundamental relationships*. https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf. University of Pisa, Slide 10. 2026.

[22] Antonio Frangioni. *Target level stepsize (vanishing)*. https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf. University of Pisa, Slide 14. 2026.

[23] Jean-Louis Goffin and Krzysztof C. Kiwiel. "Convergence of a Simple Subgradient Level Method". In: *Mathematical Programming* 85.1 (1999), pp. 207–211.

[24] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd. Springer, 2009. URL: https://hastie.su.domains/ElemStatLearn/.

[25] Guang-Bin Huang, Qin-Yu Zhu, and Chee-Kheong Siew. "Extreme learning machine: Theory and applications". In: *Neurocomputing* 70.1 (2006), pp. 489–501. DOI: 10.1016/j.neucom.2005.12.126.

[26] Hien D. Nguyen. *An Introduction to MM Algorithms for Machine Learning and Statistical*. 2016. arXiv: 1611.03969 [stat.CO]. URL: https://arxiv.org/abs/1611.03969.

[27] R. Kelley Pace and Ronald Barry. "Sparse spatial autoregressions". In: *Statistics & Probability Letters* 33.3 (1997), pp. 291–297. DOI: 10.1016/S0167-7152(96)00140-X.

[28] Lloyd N. Trefethen and David Bau. *Numerical Linear Algebra*. SIAM, 1997.